



ADVANTEST CORPORATION

SmartShell System Reference

Command Reference

SmartTest 7.0

2020-011-25

Notices

Legal Notices

All rights reserved. All text and figures included in this publication are the exclusive property of Advantest Corporation. Reproduction of this publication in any manner without the written permission of Advantest Corporation is prohibited. Information in this document is subject to change without notice.

Trademarks and Registered Trademarks

- ADVANTEST is a trademark of Advantest Corporation.
- All other marks referenced herein are trademarks or registered trademarks of their respective owners.

Warranty

The material contained in this document is provided "as is," and is subject to being changed, without notice, in future editions. Further, to the maximum extent permitted by applicable law, Advantest disclaims all warranties, either express or implied, with regard to this material and any information contained herein, including but not limited to the implied warranties of merchantability and fitness for a particular purpose. Advantest shall not be liable for errors or for incidental or consequential damages in connection with the furnishing, use, or performance of this material or of any information contained herein. Should Advantest and the user have a separate written agreement with warranty terms covering the material in this document that conflict with these terms, the warranty terms in the separate agreement shall control.

Technology Licenses

The hardware and/or software described in this document are furnished under a license and may be used or copied only in accordance with the terms of such license.

Restricted Rights Legend

If software is for use in the performance of a U.S. Government prime contract or subcontract, Software is delivered and licensed as "Commercial computer software" as defined in DFAR 252.227-7014 (June 1995), or as a "commercial item" as defined in FAR 2.101(a) or as "Restricted computer software" as defined in FAR 52.227-19 (June 1987) or any equivalent agency regulation or contract clause. Use, duplication or disclosure of Software is subject to Advantest's standard commercial license terms for 93000 systems and non-DOD Departments and Agencies of the U.S. Government will receive no greater than Restricted Rights as defined in FAR 52.227-19(c)(1-2) (June 1987). U.S. Government users will receive no greater than Limited Rights as defined in FAR 52.227-14 (June 1987) or DFAR 252.227-7015 (b)(2) (November 1995), as applicable in any technical data.

Safety Notices

CAUTION

A CAUTION notice denotes a hazard. It calls attention to an operating procedure, practice, or the like that, if not correctly performed or adhered to, could result in damage to the product or loss of important data. Do not proceed beyond a CAUTION notice until the indicated conditions are fully understood and met.

WARNING

A WARNING notice denotes a hazard. It calls attention to an operating procedure, practice, or the like that, if not correctly performed or adhered to, could result in death or severe injury. Do not proceed beyond a WARNING notice until the indicated conditions are fully understood and met.

Table of Contents

1	Test Program Development.....	9
1.1	Pattern Generation.....	9
1.1.1	Basic Pattern Generation.....	9
1.1.2	State Character Mapping Selection.....	9
1.1.3	Speed-up Vector Data Transfer.....	10
1.2	Pattern Modification.....	11
1.2.1	EDIT Vector Data.....	11
1.2.1.1	Editing One Vector.....	11
1.2.1.2	Editing Multiple Vectors.....	11
1.2.1.3	Editing Loop and Repeat Counts.....	11
1.2.2	Edit pin-based vector.....	12
1.2.3	INSERT Vector Data.....	12
1.2.4	DELETE Vector Data.....	12
1.2.5	SPLIT Pattern.....	12
1.3	Testsuite Control.....	13
1.3.1	Creating a testsuite.....	13
1.3.2	SET Values to TestMethod Parameters.....	14
1.3.3	SET Flow Variable.....	14
1.4	Test Setup Modification.....	14
1.4.1	Set Timing Edge and Level Threshold.....	14
1.4.2	Set Spec Variable Value.....	14
1.5	Test Program Query Command.....	14
1.5.1	GET FLOW and SUITE Name and Variable.....	14
1.5.2	GET TIMING EDGE and LEVEL Settings.....	14
1.5.3	GET PAT for Pattern Sequencing.....	15
1.5.4	GET SIGNAL LIST.....	15
1.5.5	GET SPEC for Level and Timing Setups.....	15
1.5.6	GET XMODE for Pin X-mode Setting.....	15
1.5.7	GET SmarTest, SmartShell and Test Program Information.....	16
1.5.8	GET Test Program.....	16
1.5.9	GET UI Report Message.....	16
1.6	Test Program Execution.....	16
1.6.1	Set Error Limit.....	16

1.6.2	Mask and Unmask Pin.....	16
1.6.3	Flow Execution.....	16
1.6.4	Suite Execution	16
1.6.5	Pattern/OpSeq Execution	16
2	Rules of Return Message	18
3	Command References.....	19
3.1	System Commands	19
3.1.1	RESET	19
3.1.2	LOAD	19
3.1.3	RELOAD	19
3.1.4	SAVE SETUP	20
3.1.5	UPDATE SETUP	20
3.1.6	GET SMARTSHELL REV	20
3.1.7	GET SMARTEST REV	21
3.1.8	GET CUR_SESSIONS.....	21
3.1.9	GET ALL_SESSIONS.....	21
3.1.10	GET CUR_SESSION	21
3.1.11	GET SESSION STATUS	22
3.1.12	GET DOCK_STATUS	22
3.1.13	GET CONNECT_STATUS.....	22
3.1.14	GET TP_PATH	23
3.1.15	GET UI_REPORT.....	23
3.1.16	GET EVENT_LOG	24
3.1.17	USE SESSION	24
3.1.18	CONNECT_DUT	24
3.1.19	DISCONNECT_DUT	25
3.1.20	CLOSE_SOCKET.....	25
3.1.21	CLOSE_SMT.....	25
3.1.22	read.....	25
3.1.23	HELP	26
3.2	Pattern Conversion and Generation.....	27
3.2.1	SET PAT CONSTR	27
3.2.2	SET SIGNAL_LIST	27
3.2.3	SET STATEMAP	28
3.2.4	SET VEC	28

3.2.5	SET TOGGLE	29
3.2.6	SET NOP	29
3.2.7	SET VEC_END	29
3.2.8	SET PAT_END	30
3.2.9	SET OPSEQ.....	30
3.3	Pattern Modification.....	31
3.3.1	EDIT VEC.....	31
3.3.2	INSERT VEC.....	32
3.3.3	DELETE VEC	32
3.3.4	EDIT VEC SIGNAL_LIST	33
3.3.5	SPLIT OPSEQ.....	34
3.4	Configure Execution.....	35
3.4.1	SET SIGNAL_MAP	35
3.4.2	SET MASK	35
3.4.3	SET UNMASK	35
3.4.4	SET ERR_LIMIT	35
3.5	Spec and Flow Variable Value.....	36
3.5.1	SET FLOW VAR	36
3.5.2	SET SPEC VAR	36
3.6	Level and Edge Settings	36
3.6.1	SET LEV.....	36
3.6.2	SET LEV_DPS	36
3.6.3	SET EDGE.....	37
3.7	TestMethod Control	38
3.7.1	SET SUITE TM_LINK.....	38
3.7.2	SET SUITE TM_VALUE	38
3.7.3	GET SUITE TM_PARAM	39
3.7.4	GET SUITE TM_PARAM_WITH_VALUE	39
3.7.5	GET SUITE TM_VALUE.....	39
3.8	Add and Remove Setups.....	41
3.8.1	ADD PAT	41
3.8.2	ADD OPSEQ.....	41
3.8.3	ADD PMF.....	41
3.8.4	ADD SETUP.....	42
3.8.5	REMOVE OPSEQ.....	42

3.9	Test Setup Query	43
3.9.1	GET BOARD_CONF	43
3.9.2	GET SIGNAL_LIST GROUP	43
3.9.3	GET SIGNAL_LIST PORT	43
3.9.4	GET PAT PORT_NAME	44
3.9.5	GET PAT SIGNAL_LIST	44
3.9.6	GET PAT LABEL_LIST MULTI_PORT	44
3.9.7	GET PAT LABEL_LIST SINGLE_PORT	44
3.9.8	GET PAT LABEL_LIST	45
3.9.9	GET PAT PRIMARY	45
3.9.10	GET PAT VEC	45
3.9.11	GET PAT VEC by SIGNAL	46
3.9.12	GET PAT VNO CYC	47
3.9.13	GET PAT TOTAL_CYC	48
3.9.14	GET PAT TOTAL_VEC	48
3.9.15	GET FLOW SUITE_LIST	49
3.9.16	GET FLOW VAR_LIST	49
3.9.17	GET FLOW VAR	49
3.9.18	GET LEV	50
3.9.19	GET LEV_DPS	50
3.9.20	GET EDGE	50
3.9.21	GET SPEC VAR_LIST	50
3.9.22	GET SPEC VAR	51
3.9.23	GET SPEC LEV EQN	51
3.9.24	GET SPEC TIM EQN	52
3.9.25	GET SPEC TIM	52
3.9.26	GET SPEC PERIOD	53
3.9.27	GET XMODE	53
3.9.28	GET DVCMAP	53
3.9.29	GET WAVETBL	54
3.10	Query Execution Results	55
3.10.1	GET FAILC	55
3.10.2	GET FAILC TAG	55
3.11	Check Command	56
3.11.1	CHK PAT	56

3.11.2	CHK MEM.....	56
3.11.3	CHK SHELL_PROTOCOL.....	56
3.12	Setup Selection	57
3.12.1	USE OPSEQ.....	57
3.12.2	USE SPEC.....	57
3.12.3	USE SETUP.....	57
3.12.4	USE SUITE.....	58
3.12.5	USE PROTOCOL	58
3.13	Execution Commands	59
3.13.1	EXE	59
3.13.2	EXE FLOW.....	59
3.13.3	EXE TO_SUITE.....	60
3.13.4	EXE SUITE	60
3.14	EDF Control	61
3.14.1	SET EDF_ON	61
3.14.2	SET EDF_OFF	61
3.14.3	GET EDF.....	61
3.15	Site Control	62
3.15.1	GET SITE_FOCUS	62
3.15.2	SET SITE_FOCUS	62
3.15.3	SET SITE_ENABLE ALL.....	62
3.15.4	SET SITE_ENABLE	62
3.15.5	SET SITE_DISABLE.....	63
3.16	Suite Control	64
3.16.1	GET SUITE SETUP <suite_name>	64
3.16.2	GET SETUP_FILE	64
3.16.3	DELETE SUITE	65
3.17	COMMAND RECORD.....	66
3.17.1	LOG CMD.....	66
3.17.2	LOG STR.....	66
3.17.3	UNLOG CMD	66

List of Figures

Fig. 1: STATEMPA Selector Type A	10
Fig. 2: STATEMPA Selector Type B	10
Fig. 3: Select Test Method pop-up Window	13
Fig. 4: Site control pop-up window before and after the command.....	63

1 Test Program Development

SmartShell commands are grouped into command groups for pattern generation and modification, test program generation and execution, testflow variable and test specification variable modification, hardware setting modification and other miscellaneous command sets.

1.1 Pattern Generation

1.1.1 Basic Pattern Generation

```
SET PAT <labelname> CONSTR <port_name>
SET SIGNAL_LIST <signal list>
SET VEC          R<n> <data>
SET VEC_END      Temporary end
SET PAT_END      <labelname>[+,]
SET OPSEQ        <spb_label> SEQ <> PORT=<> LABEL_LIST=<>[+,]
SET OPSEQ        <mpb_label> PAR LABEL_LIST=<spb_label>[+,]
```

With "SET VEC_END", the transferred vector data is held in cache and can be resumed later. With "SET PAT <> CONSTR <>" command, the data transfer of the same pattern will continue.

With "SET PAT_END", the vector data transfer is completed and it is ready for conversion.

With "SET OPSEQ SEQ", a list of one or more patterns under the same signal group will be generated and with "SET OPSEQ PAR", multiport burst label will be generated.

Example:

```
SET PAT scan_pat1 CONSTR jtag_port
SET SIGNAL_LIST pin1,pin2,pin3,pin4
SET VEC R1 0000
SET VEC R10 1010
SET VEC R1 1010
SET PAT scan_pat2 CONSTR jtag_port
SET SIGNAL_LIST pin1,pin2,pin3,pin4
SET VEC R3 0000
SET VEC R1 1010
SET PAT ctrl_pat CONSTR ctrl_port
SET SIGNAL_LIST pin5,pin6,pin7,pin8
SET VEC R1 0000
SET OPSEQ seq1 SEQ grp1 LABEL_LIST=scan_pat1 scan_pat2
SET OPSEQ seq2 SEQ grp1 LABEL_LIST=ctrl_pat
SET OPSEQ scan_test PAR LABEL_LIST=seq1 seq2
```

1.1.2 State Character Mapping Selection

It is quite often that the existing timing wave table in V93000 has multiple state maps. Currently the interface supports the following mapping:

```

PINS CLOCK
0 "{ d1:1 d2:F0N }" P
1 "{ d1:1 }" 1
2 "{ d1:0 }" 0
3 "{ d1:Z }" Z
4 "{ r1:X }" X
brk ""

STATEMAP
#physSeq statechar period selector
0 P x1 project pll1tst_d_jtag_p_12/P TS1/P
1 1 x1 project pll1tst_d_jtag_p_12/1 TS1/1
2 0 x1 project pll1tst_d_jtag_p_12/0 TS1/0
3 Z x1 project pll1tst_d_jtag_p_12/Z TS1/Z
4 X x1 project pll1tst_d_jtag_p_12/X TS1/X

```

Fig. 1: STATEMAP Selector Type A

To use pll1tst_d_jtag_p_12 mapping table, the command is

```
SET STATEMAP      pll1tst_d_jtag_p_12
```

The sample script will look like this:

```

SET PAT CONTR scan_pat
SET STATEMAP      pll1tst_d_jtag_p_12
SET SIGNAL_LIST pin1,pin2,pin3,pin4
SET VEC R1 0000
SET VEC R10 1010
SET VEC R1 1010
SET VEC R1 1010
SET VEC R1 1010

```

SmartShell interface will be enhanced to support another STATEMAP format:

```

PINS pin1
0 "{d1:0}"      0ts1 # 0
1 "{d1:1}"      1ts1 # 1
20 "{d3:0}"     0ts2 # 0
30 "{d3:1 d4:F0N}" 1ts2 # 1
brk ""

STATEMAP
#physSeq  statechars  x-mode  selector
0          0          x1      tset2
1          1          x1      tset2
20         0          x1      tset1
30         1          x1      tset1

```

Fig. 2: STATEMAP Selector Type B

Each pin or group may have different mapping, we will support this format later.

1.1.3 Speed-up Vector Data Transfer

The "SET TOGGLE" commands are used to reduce the redundancy data to be transferred over and over again when only a few data differ from the previous vector data to speed up the data transfer .

```
SET TOGGLE R1 1@0 5@H
```

The "SET NOP" command will simply tell the SmartShell server to keep adding the same vector to the pattern set by SET VEC command immediately before.

SET NOP

1.2 Pattern Modification

1.2.1 EDIT Vector Data

EDIT VEC has multiple capacities:

- 1) Editing one-line vector
- 2) Editing multiple-line vector
- 3) Editing loop and repeat count
- 4) Editing pin-based vector

1.2.1.1 Editing One Vector

```
EDIT VEC <label> <vec#>@R<#>@<data>
```

Where `vec#` is the vector position in the pattern, `R<#>` is the repeat count for this vector line, and `<data>` is vector data in order of pins which can be determined by `GET PAT SIGNAL_LIST`.

In x1-mode, it is written as for 5 pins "11111", 5 data char is mapped to 5 pins;

in x-mode, it is written as, e.g x3-mode, "111,000,111,010,111", 5 data separated by ",", mapped to 5 pins.

1.2.1.2 Editing Multiple Vectors

```
EDIT VEC <label> <vec#>@<data>&+
```

In multiple vector editing, vector repeat is kept unchanged. It will be flat vector editing and it only needs to put start vector position.

Additional vector data is appended via "&" sign.

In x1-mode for 5 pins, it will be written as 11111&00000&...

In x5-mode for 5 pins, it will be written as 111,000,111,010,111&111,000,111,010,111&...

1.2.1.3 Editing Loop and Repeat Counts

Multiple line Loop can be inserted in the middle of a flat pattern

```
EDIT VEC <label> <vec#>@R<#>@#<#>
```

Where the first "#" is the vector position, the second "#" is the number of repeats, and the third "#" is the number of vector lines to be repeated

Any existing loop structure doesn't match what the instruction looks for will issue error.

An existing loop structure matches to the instruction, only loop count will be modified.

1.2.2 Edit pin-based vector

```
EDIT VEC <label> <pin>+ <vec#>@<data>&+
```

Where <pin>+ can be one pin or several pins separated by ", ".

1.2.3 INSERT Vector Data

Occasionally additional vector data can be inserted into the pattern needed. It can be done by using INSERT command.

```
INSERT VEC [<label>] <vec#>@R<n>@data  
INSERT VEC [<label>] <vec#>@data
```

label is omitted, the primary active label will be used. <vec#> is the vector line position and <n> is the repeat count of this vector and <data> is the vector data.

1.2.4 DELETE Vector Data

Also vector data can be trimmed away with DELETE command:

```
DELETE VEC <label> <ipos>
```

<ipos> is the vector line position.

1.2.5 SPLIT Pattern

This command will extract a portion of an existing pattern from a given multiport burst label and a given port. The new label contains vector data of original pattern starting from cycle <spos> and ending at the cycle <epos> (inclusive).

```
SPLIT OPSEQ SRC=<srclabel> PORT=<> STARTC=<scyc> [STOPC=<ecyc>]
```

STOPC can be omitted.

Notes 1: the split command will work also for a loop and one or more single line repeats within the loop. However it doesn't support multiple line repeat within a loop.

Notes 2: there is a loop at start cycle or stop cycle and the distance between start cycle and stop cycle are more than 100 line vectors, the process of splitting pattern will take an excessive amount of time to complete. In this case, consider to manually flatten the pattern before the split command is used.

The command will return the new generated label or labels.

the end cycle "ecyc" is within the same pattern, the new label is named as:

```
<main_label>_<scyc>_epos
```

the end cycle "ecyc" is across multiple patterns within that port, the new generated labels will be listed as:

```
<main_labela>_<scyc>_max1  
<main_labelb>_(max1+1)_ (max2)
```

```
...
<main_labelm>_(pre_max+1)_(maxm)
<main_labeln>_(maxn+1)_(ecye)
```

Where `labela`, `labelb`, `labelm` and `labeln` etc are main label names in the burst pattern.

1.3 Testsuite Control

1.3.1 Creating a testsuite

A testsuite can be defined on the fly and used immediate afterwards:

```
SET SUITE [<suite>] TM_LINK=<tm> SPECS=<specs> OPSEQ=<label>
```

When `suite` is not provided, then a temporary suite will be created internally;

`tm` is the name of the testmethod and follows the path shown in Test Method Selection Window under SmarTest 7;

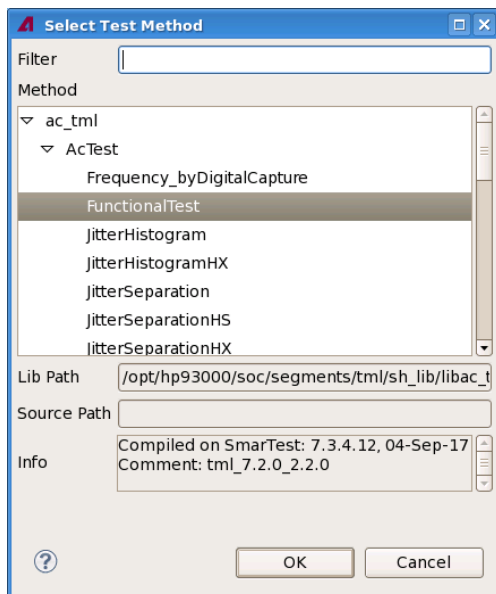


Fig. 3: Select Test Method pop-up Window

`specs` is composed of `levelset` and `timingset`.

where

`levelset` is given as `<eqn#>,<spec#>,<set#>`,

`timingset` is given as either as

`<eqn#>,<spec#>,<set#>` for set based timing or

`<timing_specification>@<set#>+`, for multiport based timing

`label` is the burst label.

1.3.2 SET Values to TestMethod Parameters

Each of the testmethod parameters can be assigned to a value or strings using the following instruction:

```
SET SUITE [<suite_name>] TM_VALUE "<parameter=value"> +  
parameter=value must be quoted.
```

1.3.3 SET Flow Variable

The same is true also for testflow variable:

```
SET FLOW VAR: SET FLOW [<flow>] VAR INT/DOUBLE/STRING "<var>=<value>"
```

1.4 Test Setup Modification

1.4.1 Set Timing Edge and Level Threshold

This command operates directly settings on the tester hardware:

```
SET LEV_DPS: SET LEV_DPS <pin>=<value>  
SET LEV: SET LEV <pin>,+ <levelkeyword>=<value>+  
SET EDGE: SET EDGE <pin>,+ (<edge>=<value>+)
```

1.4.2 Set Spec Variable Value

Values of spec variable can be changed via the following command:

```
SET SPEC VAR: SET SPEC VAR TIM/LEV <var>=<value>
```

1.5 Test Program Query Command

1.5.1 GET FLOW and SUITE Name and Variable

To query the loaded testflow information, the following group of commands are used.

```
GET FLOW SUITE_LIST: GET FLOW [<flow>] SUITE_LIST  
GET FLOW VAR: GET FLOW [<flow>] VAR [TYPE] <var>  
GET FLOW VAR_LIST: GET FLOW [<flow>] VAR_LIST [TYPE]  
GET SUITE TM_PARAM: GET SUITE [<suite_name>] TM_PARAM  
GET SUITE TM_PARAM_WITH_VALUE: GET SUITE [<suite_name>]  
TM_PARAM_WITH_VALUE  
GET SUITE TM_VALUE: GET SUITE [<testsuite>] TM_VALUE "<var>" +
```

Details can be found under the SmartShell Section 3.9

1.5.2 GET TIMING EDGE and LEVEL Settings

To query the exact timing and level setting, the following commands deal with them:

```
GET EDGE: GET EDGE <pin>  
GET LEV: GET LEV <pin>
```

1.5.3 GET PAT for Pattern Sequencing

The burst labels or operating sequences and individual patterns can be queried back:

```
GET PAT PRIMARY: GET PAT PRIMARY
GET PAT LABEL_LIST MULTI_PORT: GET PAT LABEL_LIST MULTI_PORT
GET PAT LABEL_LIST SINGLE_PORT: GET PAT LABEL_LIST SINGLE_PORT
GET PAT LABEL_LIST: GET PAT <label> LABEL_LIST <port>
```

GET Signal list and Port/Group Information

```
GET PAT PORT_NAME: GET PAT <label> PORT_NAME <pin>
GET PAT SIGNAL_LIST: GET PAT <label> SIGNAL_LIST
```

Get Total Vector and Total Cycle Counts

```
GET PAT TOTAL_CYC: GET PAT <label> TOTAL_CYC
GET PAT TOTAL_VEC: GET PAT <label> TOTAL_VEC
```

Get Vector Data and Vector Data Per Pin

```
GET PAT VEC SIGNAL: GET PAT <label> VEC <pin> <start> <stop>
GET PAT VEC: GET PAT <label> VEC <start> [STATEMAP_TABLE]
```

Get Pattern Sequencing Setting from Cycle/Vector Number

```
GET PAT VNO CYC: GET PAT <label> VNO <port> CYC=<ipos>
GET PAT VNO TAG: GET PAT <label> VNO <port> TAG=<tag>
```

1.5.4 GET SIGNAL LIST

Signal list can be defined as a group or under SmarTest 7 with a port

```
GET SIGNAL_LIST GROUP: GET SIGNAL_LIST GROUP <group>
GET SIGNAL_LIST PORT: GET SIGNAL_LIST PORT <port>
```

1.5.5 GET SPEC for Level and Timing Setups

V93000 related timing and level specification and its settings can be queried:

```
GET SPEC LEV EQN: GET SPEC [<eqn_number/eqn_name>/ALL/ACTIVE] LEV [EQN]
GET SPEC PERIOD: GET SPEC [<specification>] PERIOD <port_name/pin_name>
GET SPEC TIM EQN: GET SPEC [<eqn_number/eqn_name>/ALL/ACTIVE] TIM EQN
GET SPEC TIM: GET SPEC [ALL/ACTIVE] TIM
GET SPEC VAR: GET SPEC [<specification>] VAR [TYPE] <>
GET SPEC VAR_LIST: GET SPEC [<specification>] VAR_LIST [TIM/LEV]
```

1.5.6 GET XMODE for Pin X-mode Setting

A device cycle can use multiple timing edges called xmode and can be queried back:

```
GET XMODE: GET XMODE [<specification>] <pin_name>
```

1.5.7 GET SmarTest, SmartShell and Test Program Information

SmartShell must be always connected to one SmarTest session. The session information can be queried back:

```
GET SESSION EVENT:  GET SESSION EVENT <session_id_key>
GET SESSION STATUS:  GET SESSION STATUS <session_id_key>
GET SMARTEST REV:    GET SMARTEST REV
GET SMARTSHELL REV:  GET SMARTSHELL REV
```

1.5.8 GET Test Program

```
GET TP_PATH:  GET TP_PATH
```

1.5.9 GET UI Report Message

```
GET UI_REPORT:  GET UI_REPORT
GET EVENT_LOG:  GET EVENT_LOG
```

1.6 Test Program Execution

1.6.1 Set Error Limit

```
SET ERR_LIMIT:  SET ERR_LIMIT <number>
```

1.6.2 Mask and Unmask Pin

```
SET MASK:  SET MASK <pin>+
SET UNMASK:  SET UNMASK <pin>+
```

1.6.3 Flow Execution

EXE FLOW will execute the entire testflow

EXE TO_SUITE will execute the flow from the beginning until a specified suite

1.6.4 Suite Execution

EXE SUITE will execute a temporary testsuite or a given testsuite.

1.6.5 Pattern/OpSeq Execution

The "EXE" command can execute either main label or burst label or operating sequence.

The followings are the options used for the call:

PAT	: must present none of FAILC exists
FAILC	: failing cycle
FAILC_CNT	: total fail count of all pins
FAILC_PIN	: failing count per pin
TAG	: get vector comment text at failing cycle
MAIN=<labels>	: list of main label separated by ","

OPSEQ=<labels> : burst label or operating sequence
SIGNAL=<pinlist> : pin list from which the result will be retrieved, separated by ","
STARTC=<spos> : cycle number start from the beginning of the pattern execution
NO_OF_CYCLES=<n> : number of fails

The result can be either pass or fail

failing cycle information is needed the format will like this:

ATE_FAILC	cycle#	pin1	expected	result
ATE_FAILC	14958	gpio_78	HLLLL	LLLLL

2 Rules of Return Message

- ERROR – Normal error message.
 - ERROR: FATAL_ERROR: – Need to restart SmartShell or SmarTest.
 - ERROR: SYNTAX_ERROR: – The SmartShell command syntax is incorrect, the possible commands are listed below.
- WARNING – Normal warning message.
- Normal results string

3 Command References

3.1 System Commands

3.1.1 RESET

Description	The "RESET" command will clear all internal cache in the SmartShell server. It is needed prior the use of "USE SESSION" to connect a new SmartTest session.
Syntax	RESET
Parameters	None
Return Value	Pass
Return Error	None
Example	>>>RESET <<<pass

3.1.2 LOAD

Description	The "LOAD" command loads a specified testflow, then execute the testflow
Syntax	LOAD [TF=<testflow_name>] [TP=<testprogram_name>]
Parameters	testflow_name or testprogram_name
Return Value	Pass
Return Error	Condition: the testflow name does not exist or can not be loaded properly Return: ERROR: Failed to load the testflow. Condition: the testprogram file does not exist Return: ERROR: The prog file does not exist. Condition: the testprogram can not be loaded properly Return: ERROR: Failed to load the testprogram.
Example	>>> LOAD TF=SmartShellDemo.flw Analyzing testflow SmartShellDemo.flw... Analyzing pin configuration SmartShellDemo.cfg... Analyzing timing SmartShellDemo.tim... pass

3.1.3 RELOAD

Description	The "RELOAD" command can reload some or all of the test program components such as pin, timing, level, vector pattern or testflow, after the test program has been fully loaded and eventually the test program has fully or partially executed
Syntax	RELOAD [ALL] [LEVEL] [TIMING] [VECTOR] [TESTFLOW]
Parameters	ALL: reload all settings PIN: reload pin config LEVEL: reload levels settings TIMING: reload timing settings VECTOR: reload vector patterns TESTFLOW: reload test flow
Return Value	pass

Return Error	Condition: the parameter is wrong Return: WARNING: Unknown parameter xxx.
Example	>>>RELOAD ALL <<<DFT_RESET_ATE TESTFLOW LEVEL TIMING VECTOR <<<pass >>>RELOAD LEVEL <<<pass

3.1.4 SAVE SETUP

Description	The “ SAVE ” command saves the setup files of the given type, and reanalyzes the setup files, the relevant info will be refreshed in SmartShell server.
Syntax	SAVE SETUP [ALL] [PIN] [LEVEL] [TIMING] [VECTOR] [TESTFLOW]
Parameters	ALL : refresh all settings PIN : refresh pin config LEVEL : refresh levels settings TIMING : refresh timing settings VECTOR : refresh vector patterns TESTFLOW : refresh test flow
Return Value	pass
Return Error	None
Example	>>>SAVE SETUP LEVEL <<<pass

3.1.5 UPDATE SETUP

Description	The “ UPDATE SETUP ” command reanalyzes the setup files, the relevant info will be refreshed in SmartShell server.
Syntax	UPDATE SETUP
Parameters	NOne
Return Value	pass
Return Error	None
Example	>>>UPDATE SETUP <<<pass

3.1.6 GET SMARTSHELL REV

Description	The “ SET SMARTSHELL REV ” command returns SmartShell revision number
Syntax	GET SMARTSHELL REV
Parameters	None
Return Value	Currently used revision number of SmartShell
Return Error	None
Example	>>>GET SMARTSHELL REV <<<SmartShell Revision v3.4.4.2-20190816

3.1.7 GET SMARTEST REV

Description	The "GET SMARTEST REV" command returns SmarTest revision
Syntax	GET SMARTEST REV
Parameters	None
Return Value	Currently used revision number of SmarTest.
Return Error	None
Example	>>>GET SMARTEST REV <<<SmarTest Revision 7.4.0.2 (T), 64bit

3.1.8 GET CUR_SESSIONS

Description	The "GET CUR_SESSION" command return the connected SmarTest sessions
Syntax	GET CUR_SESSION
Parameters	None
Return Value	The connected SmarTest session
Return Error	None
Example	>>>GET CUR_SESSION <<<Session 1: ID=smartshell_demo

3.1.9 GET ALL_SESSIONS

Description	The "GET ALL_SESSIONS" command lists all the running SmarTest sessions
Syntax	GET ALL_SESSIONS
Parameters	None
Return Value	All the running SmarTest sessions
Return Error	None
Example	>>>GET ALL_SESSIONS <<<Session id

3.1.10 GET CUR_SESSION

Description	The "GET CUR_SESSION" command returns the current running SmarTest sessions connected with SmartShell
Syntax	GET CUR_SESSION
Parameters	None
Return Value	the connected running SmarTest sessions
Return Error	None
Example	>>>GET CUR_SESSION <<<Session id

3.1.11 GET SESSION STATUS

Description	The "GET SESSION STATUS" command returns SmarTest status
Syntax	GET SESSION STATUS <session_id>
Parameters	session_id: SmarTest session ID which is defined by DISPLAY_HP83000
Return Value	SmarTest is ready
Return Error	Condition: the session does not exist. Return: ERROR: No SmarTest session session_id found. Condition: the setup files are not loaded properly Return: WARNING: Please check the setup files are already loaded properly.
Example	>>>GET SESSION STATUS smartshell_demo <<<sessionID = smartshell_demo <<<SmarTest execution status for SmarTest session "smartshell_demo": READY <<<mode: OFFLINE

3.1.12 GET DOCK_STATUS

Description	The "GET DOCK_STATUS" command get the DUT board is docked mechanically to the tester head.
Syntax	GET DOCK_STATUS
Parameters	None
Return Value	DUT_DOCKED "FALSE" DUT_DOCKED "TRUE"
Return Error	None
Example	>>>GET DOCK_STATUS <<<DUT_DOCKED "FALSE"

3.1.13 GET CONNECT_STATUS

Description	The "GET CONNECT_STATUS" command checks pin electronic relays are connected
Syntax	GET CONNECT_STATUS
Parameters	None
Return Value	CONNECTED or NOT CONNECTED OR PARTIAL CONNECTED. DETAILS is provided, the it will return connection status for all pins
Return Error	None
Example	>>>GET CONNECT_STATUS <<<CONNECTED "FALSE" <<<CONNECTED PINS:BADD0,BADD1 <<<DISCONNECTED PINS:BADD2,BADD3

3.1.14 GET TP_PATH

Description	The "GET TP_PATH" command gives the test program directory path
Syntax	GET TP_PATH
Parameters	None
Return Value	Device path
Return Error	None
Example	>>>GET TP_PATH <<<output

3.1.15 GET TP LIST

Description	The "GET TP LIST" command gives all testProgame in current device path
Syntax	GET TP LIST
Parameters	None
Return Value	testprogame in current device path
Return Error	None
Example	>>>GET TP LIST <<<device path : /home/demo/xxxx/xxxx

3.1.16 GET TF LIST

Description	The "GET TF LIST" command gives all testflow in current device path
Syntax	GET TF LIST
Parameters	None
Return Value	testflo in current device path
Return Error	None
Example	>>>GET TF LIST <<<smartshell_demo.flw <<<smartshell_demo.flw.bak

3.1.17 GET UI_REPORT

Description	The "GET UI_REPORT" command gives the actual ui report window content
Syntax	GET UI_REPORT [SUITE/OPSEQ]
Parameters	SUITE:Only get current suite exe result from ui_report OPSEQ: Only get current OPSEQ exe result from ui_report
Return Value	Complete UI report text
Return Error	Condition: can not open uireport.dump Return: ERROR: Failed to get the UI info
Example	>>>GET UI_REPORT <<<xxxxxxxxxxxxxx <<<Info from UI Report <<<xxxxxxxxxxxxxx

3.1.18 GET_EVENT_LOG

Description	The "GET_EVENT_LOG" command gives the actual event log when execute a suite. This can be used after command EXE SUITE.
Syntax	GET_EVENT_LOG
Parameters	None
Return Value	Event log after suite execution
Return Error	None
Example	>>>GET_EVENT_LOG <<<xxxxxxxxxxxxx <<<Info from event log <<<xxxxxxxxxxxxx

3.1.19 USE_SESSION

Description	The "USE_SESSION" command selects from one of the existing SmarTest sessions to connect and operate on the session.
PreCondition	You can get all the running SmarTest sessions with command "GET_ALL_SESSIONS", then use "USE_SESSION" to connect one session.
Syntax	USE_SESSION=<session_id_key>
Parameters	session_id_key: SmarTest session ID which is defined by DISPLAY_HP83000
Return Value	IDLE: SmarTest starts and test program is set, but setups have not been downloaded READY: SmarTest starts and test program is loaded NOT READY: SmarTest starts but no test program is set FAILED: system errors of various kinds
Return Error	Condition: command syntax error Return: ERROR: Unsupported command syntax USE xxx Condition: the session does not ready. Return: ERROR: Failed to connect to SmarTest session <SessionID> Condition: the session does not exist. Return: ERROR: SmarTest session <SessionID> does not exist
Example	>>>USE_SESSION=smartshell_demo <<<sessionID = smartshell_demo <<<SmarTest execution status for SmarTest session "smartshell_demo": READY

3.1.20 CONNECT_DUT

Description	The "CONNECT_DUT" command sets pin electronic relays connected which behaves exactly the same as click the connect button in SmarTest Work Center menu. It should return as message issued by SmarTest.
Syntax	CONNECT_DUT

Parameters	None
Return Value	pass no error;
Return Error	ERROR: <any reason from tester>
Example	>>>CONNECT <<<pass

3.1.21 DISCONNECT_DUT

Description	The "DISCONNECT_DUT" command disconnects the pin electronic relays, which behaves exactly the same as clicking the disconnect button in the SmarTest Work Center menu.
Syntax	DISCONNECT_DUT
Parameters	None
Return Value	pass no error;
Return Error	ERROR: <any reason from tester>
Example	>>>DISCONNECT_DUT <<<pass

3.1.22 CLOSE_SOCKET

Description	The "CLOSE_SOCKET" command quits the operation. Now the server is waiting for new connection by client.
Syntax	CLOSE_SOCKET
Parameters	None
Return Value	pass no error
Return Error	ERROR: <any reason from tester>
Example	>>>CLOSE_SOCKET <<<pass

3.1.23 CLOSE_SMT

Description	"The "CLOSE_SMT" command close the SmarTest of the given session ID
Syntax	CLOSE_SMT [id]
Parameters	id: optional it is given the specified session id will be closed
Return Value	pass no error
Return Error	ERROR: <any reason from tester>
Example	>>>CLOSE_SMT xyz <<<pass

3.1.24 read

Description	The "read" command reads all supported commands and executes them
Syntax	read <SmartShell command script>
Parameters	None
Return Value	Execution messages for each of individual command

Return Error	Errir nessage if each command
Example	"smartshell_command_file" contains the following SmartShell command list: <pre> RELOAD VECTOR SET PAT aname1 CONSTR example_1 SET SIGNAL_LIST IO_000 IO_002 IO_004 IO_006 IO_008 IO_010 IO_012 IO_014 IO_016 IO_018 IO_020 SET VEC R1 000000000000 SET PAT_END aname1 STOP SET PAT aname2 CONSTR example_2 SET SIGNAL_LIST IO_001 IO_003 IO_005 IO_007 IO_009 IO_011 IO_013 IO_015 IO_017 IO_019 IO_021 SET VEC R1 HHHHHHHHHH // there is an error SET VEC R1 HHHHHHHHHH SET VEC R1 HHHHHHHHHH // there is another error SET PAT_END aname2 STOP SET PAT aname3 CONSTR example_3 SET SIGNAL_LIST tck tdi tdo tms trst_n SET VEC R3 00000 SET PAT_END aname3 STOP SET OPSEQ seq_group_name1 SEQ PORT=example_1 LABEL_LIST=aname1 SET OPSEQ seq_group_name2 SEQ PORT=example_2 LABEL_LIST=aname2 SET OPSEQ seq_group_name3 SEQ PORT=example_3 LABEL_LIST=aname3 SET OPSEQ paral_group_name PAR LABEL_LIST=seq_group_name1 seq_group_name2 seq_group_name3 </pre> >>> read <path_to_file>/smartshell_command_file

3.1.25 HELP

Description	The "HELP" command gives all supported command or each of the sub command
Syntax	HELP ALL/<command>
Parameters	None
Return Value	information of the command list or individual command
Return Error	None
Example	<pre> >>> HELP GET <<<GET ALL_SESSIONS: GET ALL_SESSIONS GET BOARD_CONF: GET BOARD_CONF PIN/PORT/GROUP GET CONNECT_STATUS: GET CONNECT_STATUS GET DOCK_STATUS: GET DOCK_STATUS GET DVCMAP: GET DVCMAP <pin_name>..... </pre>

3.2 Pattern Conversion and Generation

Note: Basic Pattern Generation, commands should be used following the steps as below.

```
SET PAT CONSTR - define the label name and relevant port
SET SIGNAL_LIST - define the pins used in the label
SET VEC - define the vector data
SET VEC_END - define the end of the label
```

3.2.1 SET PAT CONSTR

Description	The "SET PAT CONSTR" command sets the name of the pattern <label> for the following pattern generation commands including: SET SIGNAL_LIST SET STATEMAP SET VEC SET TOGGLE SET NOP SET VEC_END SET OPSEQ
Syntax	SET PAT <label> CONSTR <port>
Parameters	label: the name of the main label port : port name can be provided
Return Value	pass no error
Return Error	None
Example	>>>SET PAT new_label CONSTR jtag_port <<<pass

3.2.2 SET SIGNAL_LIST

Description	The "SET SIGNAL_LIST" command is a member command within "SET PAT CONSTR" command group and it provides the complete pin list for the "SET VEC" command. The Order of pin list is the order of vector data.
Syntax	SET SIGNAL_LIST <pin>+
Parameters	<pin>+ : list of pin name for the command group initialized by SET PAT CONSTR. It can be separated by either blank " " or comma ","
Return Value	pass no error
Return Error	Condition: the specified pins don't belong to a port Return: ERROR: (CODE_1): Can not find port from V93000 setup base on pins of DFT_SIGNALS.
Example	>>>SET SIGNAL_LIST pin1 pin2 <<<pass

3.2.3 SET STATEMAP

Description	The "SET STATEMAP" command is a member command within "SET PAT CONSTR" command group and it defines the <statemap> name used for the vector data conversion. The STATEMAP is defined in timing wavetable block <pre>PINS CLOCK 0 "{ d1:1 d2:F0N }" P 1 "{ d1:1 }" 1 2 "{ d1:0 }" 0 3 "{ d1:Z }" Z 4 "{ r1:X }" X brk "" STATEMAP #physSeq statechar period selector 0 P x1 project pll1st_d_jtag_p_12/P TS1/P 1 1 x1 project pll1st_d_jtag_p_12/1 TS1/1 2 0 x1 project pll1st_d_jtag_p_12/0 TS1/0 3 Z x1 project pll1st_d_jtag_p_12/Z TS1/Z 4 X x1 project pll1st_d_jtag_p_12/X TS1/X</pre> where "project" is a key word to tell the server the following strings are the mapping name of statemap used for pattern generation. In the example "pll1st_d_jtag_p_12" is the name that needs to be provided in SET STATEMAP command like this: SET STATEMAP pll1st_d_jtag_p_12 There can be multiple statemap names after the "project" keyword like this: <pre>STATEMAP 0 P x1 project xyz abc ...</pre> "abc" should be used, the SET STATEMAP command shall be like this: SET STATEMAP xyz
Syntax	SET STATEMAP <statemap_name>
Parameters	statemap_name: name of statemap key for the command group initialized by SET PAT CONSTR
Return Value	pass no error
Return Error	None
Example	<pre>>>>SET STATEMAP xyz <<<pass</pre>

3.2.4 SET VEC

Description	The "SET VEC" command is a member command within "SET PAT CONSTR" command group and it defines vector data and repeat count.
Syntax	SET VEC R<n> <data> [// comment string]

Parameters	R<n>: R states for the repeat keyword and n is the repeat counts data: is the vector data in x1 format which must have equal number of chars as the number of pins defined by "SET SIGNAL_LIST" command."
Return Value	pass no error
Return Error	None
Example	>>>SET VEC R1 0000 <<<pass >>>SET VEC R1 0000 // command write header <<<pass

3.2.5 SET TOGGLE

Description	The "SET TOGGLE" command is a member command within the "SET PAT CONSTR" command group and it tells the server to just change the data at the position specified.
Syntax	SET TOGGLE R<n> <pin_index>@<data>+ All bits of data will be kept the same as it is provided by the previous SET VEC command except vector data at <pin_index> position needs to be updated.
Parameters	R<n> : R states for the repeat keyword and n is the repeat counts pin_index: the pin index number defined by "SET SIGNAL_LIST" command data : the vector data in x1 format
Return Value	pass no error
Return Error	None
Example	>>>SET TOGGLE R10 3@L 4@H <<<pass

3.2.6 SET NOP

Description	The "SET NOP" command is a member command within "SET PAT CONSTR" command group and it will just repeat the previous vector for <n>.
Syntax	SET NOP R<n>
Parameters	R<n>: R states for the repeat keyword and n is the repeat counts
Return Value	pass no error
Return Error	None
Example	>>>SET NOP R10 <<<pass

3.2.7 SET VEC_END

Description	The "SET VEC_END" command is to describe the pattern data transfer has been finished.
Syntax	SET VEC_END
Parameters	None
Return Value	pass
Return Error	None

Example	>>>SET VEC_END <<<pass
----------------	---------------------------

3.2.8 SET PAT_END

Description	The "SET PAT_END" command is to describe that the pattern can be converted now.
Syntax	SET PAT_END <label>+
Parameters	The pattern name defined by SET PAT CONSTR
Return Value	pass no error
Return Error	Condition: the pattern can not be converted Return: ERROR: Fail to generate V93000 binary vector file xxxx. ERROR: Pin 'pin1', state char 'L' is used but not defined in V93000 timing setup
Example	>>>SET PAT_END pat1 pat2 <<<pass

3.2.9 SET OPSEQ

Description	The "SET OPSEQ SEQ" command creates a <seqlabel> burst label for only one group of pins under which multiple patterns <label>+ for this group of pins will be stacked up for sequential pattern execution. The "SET OPSEQ PAR command creates a <parlabel> burst label for multiple burst <label>+ labels for different pin groups defined by SET OPSEQ SEQ command."
Syntax	<pre>SET OPSEQ <seq_burst_label> SEQ PORT=<port name> LABEL_LIST=<main label list>+ SET OPSEQ <par_burst_label> PAR LABEL_LIST=<seq_burst_label>+</pre>
Parameters	<pre>seq_burst_label : main label main label list : list of main label under a given pin group or port seq burst label : sequential burst label under a given pin group or port par_burst_label : multi-port burst label name"</pre>
Return Value	pass no error
Return Error	Condition: the pattern is not defined in SmarTest Return: ERROR: Failed to generate V93000 vector for port portName, label szLabel Condition: the given burst name is too long Return: WARNING: The burst length is too long, Max length is 128, change the burst name to tmpMPB
Example	<pre>>>>SET OPSEQ jtagport_burst_label SEQ PORT=jtat_port LABEL_LIST=pat1 pat2 <<<pass >>>SET OPSEQ multiport_burst PAR LABEL_LIST=jtagport_burst_label otherport_burst_label <<<pass</pre>

3.3 Pattern Modification

3.3.1 EDIT VEC

Description	The "EDIT VEC" command changes the vector data at given vector number in the given main label
Syntax	EDIT VEC [<label>] <spos>@[[R]<n>@]<data>
Parameters	label: main label(pattern name) for vector data editing spos: the vector number R<n>: repeat counts and R key can be omitted data: state char and x-mode separated by "," and there is only one pin, then the vector data must end with "," too"
Return Value	Update data operation information , pass
Return Error	Condition: the number of arguments is incorrect Return: ERROR: SYNTAX_ERROR: - command : DFT_EDIT_VEC, argument :xxx Condition: the provided vector number is not a number Return: ERROR: the provided vector number xxx is not a number Condition: the provided vector number is out of range Return: ERROR: the provided vector number \"-1\" is not a number Condition: the pattern name that you input is not exist Return: ERROR: Pattern sLabel does not exist Condition: the pattern is a Multiport burst Return: ERROR: Pattern xxx is a Multiport burst. Condition: the pattern is a Main Label Return: ERROR: Pattern xxx is a Main Label. Condition: the provided state has no mapping in timing Return: ERROR: Can not find mapping pf state xxx Condition: the number of pins is not equal to the number of provided data Return: ERROR: The number of pins is not equal to the number of data.
Example	>>>EDIT VEC demo_jtag1 2@00H00 <<<pattern demo_jtag1 with value 00H00 <<<The mapped value is 0,0,1,0,0, <<<pass example x5-mode for 6 pins vector pattern: >>> EDIT VEC pat1 5@R2@10000,LXHLL,10010,11111,LHLHL,00000 <<<pass where it starts editing from position 5 and repeats 2 with data 100000 example x5-mode for 6 pins vector pattern: >>>EDIT VEC pat1 5@10000,LXHLL,10010,11111,LHLHL,00000 <<<pass example x5-mode for 1 pins vector pattern: >>>EDIT VEC pat1 5@10000, <<<pass where it starts editing from position 5 with data 100000

	example x1-mode for 6 pins vector pattern: <pre>>>>EDIT VEC pat1 5@101010 <<<pass</pre>
--	--

3.3.2 INSERT VEC

Description	The "INSERT VEC" command inserts the vector data at given vector number in the given label and eventually the repeat count.
Syntax	<pre>INSERT VEC [<label>] <ipos>@R<n>@data INSERT VEC [<label>] <ipos>@data</pre>
Parameters	label: main label for vector data editing spos: the vector number R<n>: repeat counts and R key can be omitted data: state char and x-mode separated by "," and only vector it must end with "," too"
Return Value	pass
Return Error	Condition: the number of arguments is not right Return: ERROR: Syntax error – command : DFT_EDIT_VEC, argument : xxxx Condition: the provided vector number is not a number Return: ERROR: the provided vector number xxx is not a number. Condition: the provided vector number is out of range Return: ERROR: the provided vector number xxx is out of range, the last vector number is xxx Condition: the provided state has no mapping in timing Return: ERROR: Can not find mapping pf state xxx Condition: the number of pins is not equal to the number of provided data Return: ERROR: The number of pins is not equal to the number od data.
Example	<pre>>>>INSERT VEC pat1 5@R5@100 <<<pass >>>INSERT VEC pat1 5@100 <<<pass</pre>

3.3.3 DELETE VEC

Description	The "DELETE VEC" deletes vector at given vector line number for a given main label
Syntax	<pre>DELETE VEC <label> <ipos></pre>
Parameters	label: main label for vector data editing ipos: the vector number
Return Value	pass
Return Error	Condition: the number of arguments is not right Return: ERROR: Syntax error – command : DFT_EDIT_VEC, argument : xxxx Condition: the provided vector number is not a number Return: ERROR: the provided vector number xxx is not a number. Condition: the provided vector number is out of range

	Return: ERROR: the provided vector number xxx is out of range, the last vector number is xxx
Example	>>>DELETE VEC jtag_patter 5 <<<pass

3.3.4 EDIT VEC SIGNAL_LIST

Description	The "EDIT VEC SIGNAL_LIST" command is pin based vector editing command at given vector line number
Syntax	EDIT VEC <label> <pin>+ <spos>@<data>&+
Parameters	label : main label for vector data editing <pin>, +: list of pin separated by comma "," spos : the start vector number epos : the end vector number data : state char and x-mode separated by "," and only one vector it must end with "," too"
Return Value	pass
Return Error	Condition: the number of arguments is not right Return: ERROR]:Syntax error – command : DFT_EDIT_VEC, argument : xxxx Condition: the provided vector number is not a number Return: ERROR: the provided vector number xxx is not a number. Condition: the provided vector number is out of range Return: ERROR: the provided vector number xxx is out of range, the last vector unnumber is xxx Condition: the provided state has no mapping in timing Return: ERROR: Can not find mapping pf state xxx Condition: the number of pins is not equal to the number of provided data Return: ERROR: The number of pins is not equal to the number od data.
Example	example x5-mode: >>>EDIT VEC label1 pin1 5@XXXXXX&XXXXXX <<<pass >>>EDIT VEC label1 pin1,pin2 5@XXXXX, <<<pass example x1-mode: >>>EDIT VEC label1 pin1 5@X&X <<<pass >>>EDIT VEC label1 pin1 5@X <<<pass

3.3.5 SPLIT OPSEQ

Description	The "SPLIT OPSEQ" command extracts portion of an existing source pattern from start to stop vector
Syntax	SPLIT OPSEQ <label> SRC=<srclabel> PRE=<prefix> PORT=<port> STARTC=<spos> STOPC=<epos>
Parameters	label : main label for vector data extraction srclabel : source multiport burst label prefix : adding a prefix to the main label port : port under which the split will begin spos : start vector number epos : stop vector number
Return Value	list of created pattern labels;
Return Error	ERROR: Syntax error, port name must be defined ERROR: Start vector number is not specified, make 0 as the default value. ERROR: Stop vector number is not specified, make the end as the default value.
Example	<pre>>>>SPLIT OPSEQ new_label SRC=old_pattern PORT=jtag_port STARTC=10 STOPC=15</pre> the ecyc end cycle is across multiple label, the new label will be listed as: <pre><label_scyc>_max1 <label>_(max1+1)_(max2) ... <label>_(pre_max+1)_(maxn) <label>_(maxn+1)_(ecye)\n SPLIT OPSEQ SRC=old_pattern PORT=jtag_port STARTC=10 STOPC=15</pre> the ecyc end cycle is across multiple label, the new label will be listed as: <pre><main_label_name_scyc>_max1 <next_main_label_name>_(max1+1)_(max2) ... <next_main_label_name>_(pre_max+1)_(maxn) <next_main_label_name>_(maxn+1)_(ecye)\n SPLIT OPSEQ SRC=old_pattern PRE=new_label PORT=jtag_port STARTC=10 STOPC=15 new_label_<main_label_name_scyc>_max1 new_label_<next_main_label_name>_(max1+1)_(max2) ... new_label_<next_main_label_name>_(pre_max+1)_(maxn) new_label_<next_main_label_name>_(maxn+1)_(ecye)"</pre>

3.4 Configure Execution

3.4.1 SET SIGNAL_MAP

Description	The "SET SIGNAL_MAP" command defines pin with a pin index number for GET FAILC and EXE FAILC command.
Syntax	SET SIGNAL_MAP <pin@index>+
Parameters	pin : the pin name of the main label index: the index to that pin name
Return Value	pass
Return Error	Condition: syntax error Return: ERROR: SYNTAX_ERROR: - command : DFT_PIN_MAP, argument : xxx
Example	>>>SET SIGNAL_MAP jtag_tdo@3 jtag_tdi@5 <<<pass

3.4.2 SET MASK

Description	The "SET MASK" command sets mask for listed pins for pattern execution
Syntax	SET MASK <pin>+
Parameters	<pin> +: list of pins separated by blank or command
Return Value	pass
Return Error	None
Example	>>>SET MASK pin pin2 <<<pass

3.4.3 SET UNMASK

Description	The "SET UNMASK" command sets unmask for listed pins for pattern execution
Syntax	SET UNMASK <pin>+
Parameters	<pin> +: list of pins separated by blank or comma
Return Value	pass
Return Error	None
Example	>>>SET UNMASK pin pin2 <<<pass

3.4.4 SET ERR_LIMIT

Description	The "SET ERR_LIMIT" command sets max error count limit for pattern result query for fail cycles
Syntax	SET ERR_LIMIT <number>
Parameters	number : max number of errors to be captured
Return Value	Max fail error set to <number>
Return Error	None
Example	>>>SET ERR_LIMIT 1000 <<<Max fail error set to 1000

3.5 Spec and Flow Variable Value

3.5.1 SET FLOW VAR

Description	The "SET FLOW VAR" command sets flow variable with the specified value.
Syntax	SET FLOW [<flow>] VAR INT/DOUBLE/STRING "<var>=<value>"
Parameters	flow : name of the test flow var : variable name value: value"
Return Value	pass
Return Error	None
Example	>>>SET FLOW VAR INT var1=0.5 <<<pass

3.5.2 SET SPEC VAR

Description	The "SET SPEC VAR" command sets primary specification or the given specification's variable to value.
Syntax	SET SPEC [<specname>] VAR TIM/LEV <var>=<value>
Parameters	specname : spec name optional var : variable name value : value
Return Value	pass no error
Return Error	Condition: The spec does not exist. Return: ERROR: Level/Timing spec xxx does not exist!
Example	>>>SET SPEC VAR LEV Vcoef=0.5 <<<pass

3.6 Level and Edge Settings

3.6.1 SET LEV

Description	The "SET LEV" command sets level value for such as vol, voh, vil, vih etc
Syntax	SET LEV <pin>, + <levelkeyword>=<value>+
Parameters	pin : pin name or list of pins separated by comma "," levelkeyword: such vih, vil, vol, vol etc value : value in mV"
Return Value	pass
Return Error	WARNING: Invalid level param xxx! WARNING: The mode of pin xxx is OFF, can not be reset!
Example	>>>SET LEV pin,pin2 vih=1000 vil=300 <<<pass

3.6.2 SET LEV_DPS

Description	The "SET LEV_DPS" command sets DPS pin's data
--------------------	---

Syntax	SET LEV_DPS <pin>=<value>
Parameters	pin: DPS pin name value: value in mV
Return Value	pass
Return Error	Condition: the specified pin does not exist Return: ERROR: Pin xxx is not A DPS pin
Example	>>>SET LEV_DPS VDD 0.9 <<<pass

3.6.3 SET EDGE

Description	The "SET EDGE" command sets primary specification or the given specification timing edge position
Syntax	SET EDGE <pin>, + (<edge>=<value>+) The specification of position string needs be encapsulated with ()
Parameters	pin : pin name or list of pins separated by comma",", edge : d#, r# or e# value : value in ns"
Return Value	pass no error
Return Error	Condition: the specified pin is not defined in the original timing waveform Return: ERROR: xxx was not defined in the original timing waveform!
Example	>>>SET EDGE pin1 (d1=1 d2=2.5) <<<pass >>>SET EDGE pin1,pin2 (d1=1 d2=2.5) <<<pass

3.7 TestMethod Control

3.7.1 SET SUITE TM_LINK

Description	The "SET SUITE TM_LINK" command creates a floating testsuite for a given testmethod and specs as well as opseq.
Syntax	SET SUITE [<suite_name>] TM_LINK=<tm> SPEC=<levelset>&<timingset> OPSEQ=<label> SET SUITE [<suite_name>] TM_LINK=<tm> SPEC=<specs> OPSEQ=<opseq>
Parameters	SET SUITE [<suite_name>] TM_LINK=mytm SPEC=<levelset>&<timingset> OPSEQ=<label> suite_name: this is not provided, then internally a tmpsuite will be created mytm : testmethod name such tml.DC.General_PMU levelset : <eqnset>,<specname>,<setno> timingset : <eqnset>,<specname>,<setno> or specname@<setno>,+ label : either multi-port burst label or single-port burst label or main label depends on timing specs
Return Value	Pass
Return Error	Condition: the TestMethod can not be found. Return: ERROR: No valid Test method specified. Condition: the given pattern does not exist. ERROR: Pattern xxx does not exist.
Example	>>>SET SUITE TM_LINK=mytm SPEC=99,1,1&specjtab@1,1,1 OPSEQ=label <<<pass This will use the parameter as primary setups for testsuite >>>SET SUITE TM_LINK=mytm SPEC=99,1,1&() OPSEQ=() <<<pass This will clear the existing timing and label to null >>>SET SUITE TM_LINK=mytm SPEC=99,1,1& <<<pass This will use existing timing and label as primary setup. setup parameters are ignored, then the execution will rely on testmethod for specification and operating sequence

3.7.2 SET SUITE TM_VALUE

Description	The "SET SUITE TM_VALUE" command assigns for a floating testsuite or a given testsuite a value to testmethod parameter.
Syntax	SET SUITE [<suite_name>] TM_VALUE <"parameter=value"> +
Parameters	Quote around "parameter=value" are needed. suite_name: this is not provided, it will use internally created tmpsuite which was created by SET SUITE TM_LINK parameter : testmethod parameter name value : value to be assigned to"
Return Value	pass

Return Error	Condition: the parameter does not exist and so on Return: ERROR: Can not set up the parameter properly
Example	>>>SET SUITE TM_VALUE "var1=0.3"8 "var2=0.8 <<<pass

3.7.3 GET SUITE TM_PARAM

Description	The "GET SUITE TM_PARAM" command gives suite <testsuite> testmethod parameter list names
Syntax	GET SUITE [<suite_name>] TM_PARAM
Parameters	suite_name: testsuite name
Return Value	var1 var2
Return Error	Condition: the suite name is not defined in test flow. Return: ERROR: The suite name may not be defined in testflow. Condition: the parameter does not exist and so on Return: ERROR: Can not query the parameter properly
Example	<i>From the floating testsuite:</i> >>>GET SUITE TM_PARAM <i>From an existing testsuite:</i> >>>GET SUITE pll_test TM_PARAM

3.7.4 GET SUITE TM_PARAM_WITH_VALUE

Description	The "GET SUITE TM_PARAM_WITH_VALUE" command gives suite <testsuite> testmethod parameter list names and values
Syntax	GET SUITE [<suite_name>] TM_PARAM
Parameters	suite_name: testsuite name
Return Value	var1=value1 var2=value2
Return Error	Condition: the suite name is not defined in test flow. Return: ERROR: The suite name may not be defined in testflow. Condition: the parameter does not exist and so on Return: ERROR: Can not query the parameter properly
Example	<i>From the floating testsuite:</i> >>>GET SUITE TM_PARAM <i>From an existing testsuite:</i> >>>GET SUITE pll_test TM_PARAM

3.7.5 GET SUITE TM_VALUE

Description	The "GET SUITE TM_VALUE" command gives the value of a testmethod parameter in a test suite
Syntax	GET SUITE [<testsuite>] TM_VALUE "<var>" +
Parameters	testsuite: testsuite name var : testmethod parameter name Notes : need quote " surrounding each variable name

Return Value	<var>=value/parameter
Return Error	None
Example	<i>From the floating testsuite:</i> <pre>>>>GET SUITE TM_VALUE "settlingtime"</pre> <i>From an existing testsuite:</i> <pre>>>>GET SUITE pll_test TM_VALUE "settlingtime" "output"</pre>

3.8 Add and Remove Setups

3.8.1 ADD PAT

Description	The "ADD PAT" command adds new <pattern file> to the existing pattern list in SmarTest.
Syntax	ADD PAT <newpatternlistfile>
Parameters	newpatternfile: pattern with path to be added
Return Value	pass no error
Return Error	ERROR: Fail to load binary vector file from pat_path" ERROR: File xxx does not exist" ERROR: Can not open pattern file xxx" ERROR: The same pattern file exist"
Example	>>>ADD OPSEQ /home/demo/vectors/newlabel.binl <<<pass >>>ADD OPSEQ /home/demo/vectors/newlabel <<<pass

3.8.2 ADD OPSEQ

Description	The "ADD OPSEQ" command adds new <burst file> to the existing pattern list in SmarTest.
Syntax	ADD OPSEQ <newburstfile>
Parameters	newburstfile: burst with path to be added
Return Value	pass no error
Return Error	ERROR: Fail to load binary vector file from pat_path" ERROR: File xxx does not exist" ERROR: Can not open pattern file xxx" ERROR: The same pattern file exist"
Example	>>>ADD OPSEQ /home/demo/vectors/newburst <<<pass

3.8.3 ADD PMF

Description	The "ADD PMF" command adds new <pattern master file> to the existing pattern list in SmarTest.
Syntax	ADD PMF <newpatternmasterfile>
Parameters	newpatternmasterfile: pattern master file with path to be added
Return Value	pass no error
Return Error	ERROR: Fail to load binary vector file from pat_path" ERROR: File xxx does not exist" ERROR: Can not open pattern file xxx" ERROR: The same pattern file exist"
Example	>>>ADD OPSEQ /home/demo/vectors/newlabel.pmf1 <<<pass

3.8.4 ADD SETUP

Description	The "ADD SETUP" command adds timing setup to the existing master file
Syntax	ADD SETUP <specsfiles>
Parameters	specsfiles: specification file to be added to the master file
Return Value	DFT_TIM_APPEND Finished
Return Error	None
Example	>>>ADD SETUP /home/demo/timing/subfold/timing1.setup <<<pass

3.8.5 REMOVE OPSEQ

Description	The "REMOVE OPSEQ" command removes <label> from the pattern list in the test program, the file itself remains untouched.
Syntax	REMOVE OPSEQ <label>
Parameters	label: multi-port burst label or single-port burst label or main label
Return Value	pass no error
Return Error	Condition: the label does not exist Return: ERROR: Can not find label xxx at any port. Please double check.
Example	>>>REMOVE OPSEQ jtag_temp_pattern <<<pass

3.9 Test Setup Query

3.9.1 GET BOARD_CONF

Description	The "GET BOARD_CONF" command gives pin configuration
Syntax	GET BOARD_CONF PIN/PORT/GROUP
Parameters	PIN : list all pins PORT : list all ports GROUP: list all groups
Return Value	Complete list of signals, ports or pin groups
Return Error	None
Example	>>>GET BOARD_CONF PIN <<<"BADD0 I BADD1 I BADD2 I >>>GET BOARD_CONF PORT <<<"EXAMPLE1 EXAMPLE2 EXAMPLE3....." >>>GET BOARD_CONF GROUP <<<"GROUP1 GROUP2 GROUP3....."

3.9.2 GET SIGNAL_LIST GROUP

Description	The "GET SIGNAL_LIST GROUP" command gives complete signal list from the given group
Syntax	GET SIGNAL_LIST GROUP <group>
Parameters	group : name of pin group
Return Value	Signal list
Return Error	Condition: the group does not exist Return: ERROR Group name xxx is invalid!
Example	>>>GET SIGNAL_LIST GROUP test_pin_group <<<"PIN1 PIN2 PIN3 PIN4 PIN5 PIN6....."

3.9.3 GET SIGNAL_LIST PORT

Description	The "GET SIGNAL_LIST PORT" command gives complete signal list from the <port>
Syntax	GET SIGNAL_LIST PORT <port>
Parameters	group: name of pin group
Return Value	Signal list
Return Error	Condition: the port does not exist Return: ERROR: Group name xxx is invalid!
Example	>>>GET SIGNAL_LIST PORT TEST_PORT <<<"PIN1 PIN2 PIN3 PIN4 PIN5....."

3.9.4 GET PAT PORT_NAME

Description	The "GET PAT PORT_NAME" command gives port name from a given pin in a given multi-port label
Syntax	GET PAT <label> PORT_NAME <pin>
Parameters	label: multi-port burst label pin : a pin
Return Value	Port name of the specified pin
Return Error	Condition: the label does not exist Return: ERROR: Pattern xxx does not exist.
Example	>>>GET PAT demo_pat_group PORT_NAME pin1 <<<PORT1

3.9.5 GET PAT SIGNAL_LIST

Description	The "GET PAT SIGNAL_LIST" command gives signal list in a given <label>
Syntax	GET PAT <label> SIGNAL_LIST
Parameters	label: main label
Return Value	Signal list
Return Error	Condition: the label does not exist Return: ERROR: Pattern xxx does not exist. Condition: the label is a multiport burst Return: Error Pattern xxx is a Multiport burst.
Example	>>>GET PAT demo_pat_group SIGNAL_LIST <<<PIN1 PIN2 PIN3 PIN4 PIN5.....

3.9.6 GET PAT LABEL_LIST MULTI_PORT

Description	The "GET PAT LABEL_LIST MULTI_PORT" command gives all multi-port burst labels and associated ports
Syntax	GET PAT LABEL_LIST MULTI_PORT
Parameters	None
Return Value	Label list
Return Error	None
Example	>>>GET PAT LABEL_LIST MULTI_PORT <<<LABEL1 LABEL2 LABEL3

3.9.7 GET PAT LABEL_LIST SINGLE_PORT

Description	The "GET PAT LABEL_LIST SINGLE_PORT" command gives all single-port (@) burst labels
Syntax	GET PAT LABEL_LIST SINGLE_PORT
Parameters	None
Return Value	Label list
Return Error	WARNING: no results is found

Example	>>>GET PAT LABEL_LIST SINGLE_PORT <<<example_pat
----------------	---

3.9.8 GET PAT LABEL_LIST

Description	The "GET PAT LABEL_LIST" command gives MAIN pattern name list for single port or for multi-port (a given main port)
Syntax	GET PAT <label> LABEL_LIST <port>
Parameters	label: multi-port burst label name port : port name
Return Value	List of main pattern labels
Return Error	Condition: the busrt does not exist Return: ERROR: Pattern xxx does not exist Condition: the port does not exist. Return: ERROR: Port xxx might not exist
Example	>>>GET PAT multi_port_burst_label LABEL_LIST demo_port <<<pat1 demo_port <<<pat2 demo_port

3.9.9 GET PAT PRIMARY

Description	The "GET PAT PRIMARY" command gives the primary label name
Syntax	GET PAT PRIMARY
Parameters	None
Return Value	Active label name
Return Error	None
Example	>>>GET PAT PRIMARY <<<primary_label1

3.9.10 GET PAT VEC

Description	The "GET PAT VEC" command gives vector data at a given vector number
Syntax	GET PAT <label> VEC <vec_number>
Parameters	label: main label vec_number: the vector number
Return Value	<data>[, +] where <data> is vector data separated by “,” and “&”. In x1 mode, “&” will be omitted and no blank in between. <i>Example:</i> 00000 which can either be 1 x5- or 5 x1-vector data. Issue command GET XMODE to figure it out. <i>Example:</i> 00000,00000 which shows 1 x5-vector data of 2 pin data.
Return Error	Condition: the label doesn't exist

	Return: Error: Pattern xxx does not exist. Condition: the vector number exceeds the limit. Return: Error: the provided vector number xxxx is out of range, the last vector number is <max number>.
Example	<pre>>>>GET PAT demo_pat VEC 5 <<<00L00 1</pre>

3.9.11 GET PAT VEC by SIGNAL

Description	The "GET PAT VEC by SIGNAL" command gives vector data from start vector to stop vector for a given signal
Syntax	GET PAT <label> VEC <pin> <start> <stop>
Parameters	label: main label name pin : pin start: start vector number stop : stop vector number"
Return Value	<data>[,+[&]] where <data> is vector data separated by ",", " and "&". In x1 mode, "&" will be omitted and no blank in between. Example: 00000 which can either be 1 x5- or 5 x1-vector data. Issue command GET XMODE to figure it out. Example: 00000&00000&10101 which shows 3 x5-vector data of 1 pin data. pin does not belong to the given main pattern pin list, error: ERROR: "<pin>" doesn't belong to the given main pattern label "<label>" <stop> is outside of max available vector range, error: ERROR: "<stop>" is outside of max available vector range the physical waveform index PW <number> from pattern <label> is not found in the wave table, error: ERROR: "<name of the wave table>" doesn't contain the mapping for waveform index "<number>" given by pattern label "<label>" NOTES 1) pattern port and wave port can be different; 2) pin in pattern port can be in a timing port with different port name
Return Error	ERROR: "the label <label> doesn't contain the provided pin <pin> under the port <xxxxxx> ERROR: "<stop>" vector number <xxxx> is outside of max available vector range, the last vector number is <max_number> ERROR: "<name of the wave table>" doesn't contain the mapping for waveform index "<number>" given by pattern label "<label>"

Example	>>>GET PAT demo_pat_group VEC pin1 5 10 <<<LLLLLL
----------------	--

3.9.12 GET PAT VNO CYC

Description	The "GET PAT VNO CYC" command gives vector info from the given port and given cycle number. It supports also nested loop which contains one or more single-line repeat(s). It doesn't support nested loop which contains repeat block containing multiple lines of vector again. The following figures show the two pattern cases. If the cycle falls here, it supports <table><tr><th>Vector#</th><th>Instruction</th><th>Comment</th><th></th></tr><tr><td>0</td><td></td><td>0 Z X 1 0</td><td></td></tr><tr><td>1</td><td></td><td>1 Z X 1 0</td><td></td></tr><tr><td>2</td><td></td><td>0 Z X 1 1</td><td></td></tr><tr><td>3</td><td></td><td>1 Z X 1 1</td><td></td></tr><tr><td>4</td><td></td><td>0 Z X 1 1</td><td></td></tr><tr><td colspan="4">- LOOP 10</td></tr><tr><td>5</td><td></td><td>1 Z X 1 1</td><td></td></tr><tr><td>6</td><td></td><td>0 Z X 1 1</td><td></td></tr><tr><td>7</td><td>RPTV 1 100</td><td>1 Z X 1 1</td><td></td></tr><tr><td>8</td><td></td><td>0 Z X 1 1</td><td></td></tr><tr><td>9</td><td></td><td>1 Z X 1 1</td><td></td></tr><tr><td>10</td><td></td><td>0 Z X 1 1</td><td></td></tr><tr><td>11</td><td></td><td>1 Z X 1 1</td><td></td></tr><tr><td>12</td><td></td><td>0 Z X 0 1</td><td></td></tr><tr><td>13</td><td></td><td>1 Z X 0 1</td><td></td></tr><tr><td>14</td><td></td><td>0 Z X 1 1</td><td></td></tr><tr><td colspan="4">- LOOPEND</td></tr><tr><td>15</td><td></td><td>1 Z X 1 1</td><td></td></tr></table> If the cycle falls here, it does not support <table><tr><th>Vector#</th><th>Instruction</th><th>Comment</th><th></th></tr><tr><td>0</td><td></td><td>0 Z X 1 0</td><td></td></tr><tr><td>1</td><td></td><td>1 Z X 1 0</td><td>Reset via TR...</td></tr><tr><td>2</td><td></td><td>0 Z X 1 1</td><td></td></tr><tr><td>3</td><td></td><td>1 Z X 1 1</td><td></td></tr><tr><td>4</td><td></td><td>0 Z X 1 1</td><td></td></tr><tr><td colspan="4">- LOOP 10</td></tr><tr><td>5</td><td></td><td>1 Z X 1 1</td><td></td></tr><tr><td>6</td><td></td><td>0 Z X 1 1</td><td></td></tr><tr><td>7</td><td>RPTV 3 100</td><td>1 Z X 1 1</td><td></td></tr><tr><td>8</td><td></td><td>0 Z X 1 1</td><td></td></tr><tr><td>9</td><td></td><td>1 Z X 1 1</td><td></td></tr><tr><td colspan="4">- REPEATEND</td></tr><tr><td>10</td><td></td><td>0 Z X 1 1</td><td></td></tr><tr><td>11</td><td></td><td>1 Z X 1 1</td><td></td></tr><tr><td>12</td><td></td><td>0 Z X 0 1</td><td>Test-Logic-R...</td></tr><tr><td>13</td><td></td><td>1 Z X 0 1</td><td></td></tr><tr><td>14</td><td></td><td>0 Z X 1 1</td><td>Run-Test/Idle</td></tr><tr><td colspan="4">- LOOPEND</td></tr><tr><td>15</td><td></td><td>1 Z X 1 1</td><td></td></tr></table>	Vector#	Instruction	Comment		0		0 Z X 1 0		1		1 Z X 1 0		2		0 Z X 1 1		3		1 Z X 1 1		4		0 Z X 1 1		- LOOP 10				5		1 Z X 1 1		6		0 Z X 1 1		7	RPTV 1 100	1 Z X 1 1		8		0 Z X 1 1		9		1 Z X 1 1		10		0 Z X 1 1		11		1 Z X 1 1		12		0 Z X 0 1		13		1 Z X 0 1		14		0 Z X 1 1		- LOOPEND				15		1 Z X 1 1		Vector#	Instruction	Comment		0		0 Z X 1 0		1		1 Z X 1 0	Reset via TR...	2		0 Z X 1 1		3		1 Z X 1 1		4		0 Z X 1 1		- LOOP 10				5		1 Z X 1 1		6		0 Z X 1 1		7	RPTV 3 100	1 Z X 1 1		8		0 Z X 1 1		9		1 Z X 1 1		- REPEATEND				10		0 Z X 1 1		11		1 Z X 1 1		12		0 Z X 0 1	Test-Logic-R...	13		1 Z X 0 1		14		0 Z X 1 1	Run-Test/Idle	- LOOPEND				15		1 Z X 1 1	
Vector#	Instruction	Comment																																																																																																																																																											
0		0 Z X 1 0																																																																																																																																																											
1		1 Z X 1 0																																																																																																																																																											
2		0 Z X 1 1																																																																																																																																																											
3		1 Z X 1 1																																																																																																																																																											
4		0 Z X 1 1																																																																																																																																																											
- LOOP 10																																																																																																																																																													
5		1 Z X 1 1																																																																																																																																																											
6		0 Z X 1 1																																																																																																																																																											
7	RPTV 1 100	1 Z X 1 1																																																																																																																																																											
8		0 Z X 1 1																																																																																																																																																											
9		1 Z X 1 1																																																																																																																																																											
10		0 Z X 1 1																																																																																																																																																											
11		1 Z X 1 1																																																																																																																																																											
12		0 Z X 0 1																																																																																																																																																											
13		1 Z X 0 1																																																																																																																																																											
14		0 Z X 1 1																																																																																																																																																											
- LOOPEND																																																																																																																																																													
15		1 Z X 1 1																																																																																																																																																											
Vector#	Instruction	Comment																																																																																																																																																											
0		0 Z X 1 0																																																																																																																																																											
1		1 Z X 1 0	Reset via TR...																																																																																																																																																										
2		0 Z X 1 1																																																																																																																																																											
3		1 Z X 1 1																																																																																																																																																											
4		0 Z X 1 1																																																																																																																																																											
- LOOP 10																																																																																																																																																													
5		1 Z X 1 1																																																																																																																																																											
6		0 Z X 1 1																																																																																																																																																											
7	RPTV 3 100	1 Z X 1 1																																																																																																																																																											
8		0 Z X 1 1																																																																																																																																																											
9		1 Z X 1 1																																																																																																																																																											
- REPEATEND																																																																																																																																																													
10		0 Z X 1 1																																																																																																																																																											
11		1 Z X 1 1																																																																																																																																																											
12		0 Z X 0 1	Test-Logic-R...																																																																																																																																																										
13		1 Z X 0 1																																																																																																																																																											
14		0 Z X 1 1	Run-Test/Idle																																																																																																																																																										
- LOOPEND																																																																																																																																																													
15		1 Z X 1 1																																																																																																																																																											
Syntax	GET PAT <multiport_label> VNO <port> CYC=<ipos>																																																																																																																																																												
Parameters	multiport_label : multi-port burst label name port : port ipos : cycle number"																																																																																																																																																												
Return Value	The vector info of the given cycle																																																																																																																																																												
Return Error	Condition: the label is not a multiport burst Return: ERROR: Pattern demo_jtag1 is a Main Label Condition: the label does not exist Return: ERROR: Pattern jtag_demo11 does not exist Condition: the port does not exist Return: ERROR: The specified port xxx does not exist																																																																																																																																																												
Example	>>>GET PAT demo_jtag1 VNO pJTAG CYC=5 <<<demo_jtag1:5 L0 R1 0 main_vector_label_name1 ... main_vector_label_nameX:#vecnum L# R# p n L# R# m where #vecnum is the vector number in the main pattern L# : >1 there are multiple lines of vector repeat R# : total repeat count p : relative vector number within multiple lines of vector repeat block n : number of loop counts to get to this cycle for multiple lines of vector repeat m : number of repeat counts to get this cycle in a single line of vector repeat Example:																																																																																																																																																												

	<pre>label_1_X5 label_2_X5:10665 L0 R1 0</pre> Explanation: the given cycle number is associated with main label "label_2_X5" L0 means no multiple lines of vectors for the repeat R1 means one single vector 0 means it is at the 0 position of the vector line Example: <pre>>>> GET PAT mpb_label1 VNO port1 CYC=11150 <<< label_1_Y7 label_2_Y5:10 L10 R14 4 10 L0 R119 20</pre> Explanation: the given cycle number is located at main label "label_2_Y5" L10 means 10-line vectors repeat R14 means the 10-line vectors will repeat 14 times 4 means the given cycle is at 4th vector position in this 10-line vectors 10 means the 10-line vectors at the 10th loops L0 means there is a single line vector repeat R119 means this line of vector repeats for 119 20 means this cycle is at 20th repeat count So there could be of total 119+9=128 cycles within this repeat block there is only one single-line repeat. To figure it out exactly how many single-line repeat, we can use GET PAT VEC command to figure it out
--	--

3.9.13 GET PAT TOTAL_CYC

Description	The "GET PAT TOTAL_CYC" command gives total cycle number in a given main label
Syntax	GET PAT <label> TOTAL_CYC
Parameters	label : main label name
Return Value	total cycle number
Return Error	Condition: the label does not exist Return: ERROR: Pattern xxx does not exist Condition: the label is not a multiport burst Return: ERROR: Pattern xxx is a Multiport burst
Example	<pre>>>>GET PAT demo_jtag1 TOTAL_CYC <<<32</pre>

3.9.14 GET PAT TOTAL_VEC

Description	The "GET PAT TOTAL_VEC" command gives total vector number in a given main label.
Syntax	GET PAT <label> TOTAL_VEC
Parameters	label: main label name
Return Value	Condition: the label does not exist Return: ERROR: Pattern xxx does not exist

	Condition: the label is not a multiport burst Return: ERROR: Pattern xxx is a Multiport burst
Return Error	None
Example	>>>GET PAT demo_jtag1 TOTAL_VEC <<<32

3.9.15 GET FLOW SUITE_LIST

Description	The "GET FLOW SUITE_LIST" command gives suite list in a current working flow or a given flow.
Syntax	GET FLOW [<flow>] SUITE_LIST
Parameters	flow: flow name
Return Value	list of all suites under this testflow <flow>
Return Error	None
Example	>>>GET FLOW SUITE_LIST <<<test1 <<<test2 <<<jtag_demo

3.9.16 GET FLOW VAR_LIST

Description	The "GET FLOW VAR_LIST" command gives all variables in the current working flow
Syntax	GET FLOW VAR_LIST TYPE
Parameters	TYPE: one of STRING/DOUBLE/INT
Return Value	list of all variables
Return Error	None
Example	>>>GET FLOW VAR_LIST INT <<<"var1 <<<var2 <<<"

3.9.17 GET FLOW VAR

Description	The "GET FLOW VAR" command gives variable value in the current working flow
Syntax	GET FLOW VAR TYPE <var>
Parameters	TYPE: one of STRING/DOUBLE/INT var : variable name"
Return Value	value of the testflow variable
Return Error	None
Example	>>>GET FLOW VAR STRING var1 <<< var1 "hello smartshell"

3.9.18 GET LEV

Description	The "GET LEV" command gives level setup for a given <pin>
Syntax	GET LEV <pin>
Parameters	pin: pin name
Return Value	Pin=xxx vil=0 vih=1.8 vol=0.5 voh=0.7 iol=0.1 ioh=0.2.....
Return Error	None
Example	>>>GET LEV test_pin1 <<<pin=test_pin1 vil=0 vih=1.8 vol=0.5 voh=0.7 vcl=-2 vch=7 iol=0.1 ioh=0.2

3.9.19 GET LEV_DPS

Description	The "GET LEV_DPS" command gives level DPS pins's value
Syntax	GET LEV_DPS [<pin>]
Parameters	pin: pin name
Return Value	Pin value
Return Error	Condition: pin is not a DPS pin Return: ERROR: Pin xxx is not A DPS pin Condition: pin doesn't exist Return: ERROR: Pin xxx does not exist
Example	>>>GET LEV_DPS VDD <<<VDD 0.8

3.9.20 GET EDGE

Description	The "GET EDGE" command gives timing edge setup for a given pin
Syntax	GET EDGE <pin>
Parameters	pin: pin name
Return Value	pin_name d1:0 d2:1
Return Error	Condition: no edge information is stored Return: WARNING: no results is found
Example	>>>GET EDGE TDO <<<TDO d1:0 r1:14

3.9.21 GET SPEC VAR_LIST

Description	The "GET SPEC VAR_LIST" command gives all spec variable names in the current specification or a given specification
Syntax	GET SPEC [<specification/eqn_no, spec_no>] VAR_LIST [TIM/LEV]
Parameters	specification: specification name for multi-port eqn_no,spec_no: equation number and spec number
Return Value	all spec variables of given type
Return Error	Condition: no primary level/timing used

	Return: ERROR: No level used Return: ERROR: No timing used
Example	<pre>>>>GET SPEC VAR_LIST TIM <<<E9 Frequency_1 >>>GET SPEC VAR_LIST LEV <<<spec_var1 spec_var2 spec_var3</pre>

3.9.22 GET SPEC VAR

Description	The "GET SPEC VAR" command gives spec variable value in the current specification or a given specification.
Syntax	GET SPEC [<specification/eqn_no,spec_no>] VAR <type> <var>
Parameters	type: TIM/LEV Var: the variable specification: specification name for multi-port eqn_no,spec_no: equation number and spec number
Return Value	value of the spec variable
Return Error	Condition: <spec_var> is not found. Result: ERROR: occurred in SVLR? LEV, PRM, ,"spec_var
Example	<pre>>>>GET SPEC VAR LEV spec_var <<<DFT_QUERY_ATE_INFO SPEC VAR LEV spec_var "2.5"</pre>

3.9.23 GET SPEC LEV EQN

Description	The "GET SPEC LEV [EQN]" command gives primary level set for all or the current one
Syntax	GET SPEC [<eqn_number/eqn_name>/ALL/ACTIVE] LEV [EQN]
Parameters	eqn_number : equation number
Return Value	EQN is given, it will return equation If... EQNSET 5: "DC_TEST" LEVSET 3: "DC_Strict_in_Relax_out" SPECSET 2: level_spec_low
Return Error	None
Example	<pre>>>>GET SPEC ALL LEV EQN EQNSET 1: "Digital" LEVSET 1: "No Active Termination" SPECSET 1: level_spec1 >>>GET SPEC 1 LEV EQN</pre>

	<pre> <<<EQNSET 1: "Digital" SPECS spec_var1 spec_var2 EQUATIONS LEVELSET 1 "No Active Termination" PINS pin1 pin2 pin3 vil = 0 vih = vdd_0 vol = vdd_0 / 2.0 - 0.05 voh = vdd_0 / 2.0 + 0.05 >>>GET SPEC ACTIVE LEV EQN <<<EQNSET 1: "Digital" LEVSET 1: "No Active Termination" SPECSET 2: level_spec_low </pre>
--	---

3.9.24 GET SPEC TIM EQN

Description	The "GET SPEC TIM EQN" command gives primary timing EQNSET or a given name.
Syntax	GET SPEC [<eqn_number/eqn_name>/ALL/ACTIVE] TIM EQN
Parameters	None
Return Value	EQNSET 1: "PLL_TEST" TIMINGSET 1: "PLL_out_test"
Return Error	None
Example	<pre> >>>GET SPEC ALL TIM EQN <<< EQNSET 12: "Tim_eqn_clk_EQNSET" TIMSET 1: "tset2" SPECSET 1: Tim_Spec1 EQNSET 13: "Tim_eqn_static_EQNSET" TIMSET 1: "tset2" SPECSET 1: Tim_Spec1 EQNSET 14: "Tim_eqn_reg_X5_EQNSET" TIMSET 1: "tset2" SPECSET 1: Tim_Spec1 >>>GET SPEC ALL TIM <<<"timing_spec_3_ports_short" "timing_spec_3_ports_full" "timing_spec_3_ports_part" >>>GET SPEC ACTIVE TIM EQN <<<timing_spec_3_ports_part <<<PORT: GROUP_CLK EQNSET 12: "Tim_eqn_clk_EQNSET" TIMSET 1: "tset2" SPECSET "timing_spec_3_ports_part" <<<PORT: GROUP_REST EQNSET 13: "Tim_eqn_static_EQNSET" TIMSET 1: "tset2" SPECSET "timing_spec_3_ports_part" <<<PORT: GROUP_MAIN EQNSET 14: "Tim_eqn_reg_X5_EQNSET" TIMSET 1: "tset2" SPECSET "timing_spec_3_ports_part" </pre>

3.9.25 GET SPEC TIM

Description	The "GET SPEC TIM" command gives timing EQNSET or a given name
Syntax	GET SPEC [ALL/ACTIVE] TIM

Parameters	None
Return Value	Specification w/ [port: pinlist]
Return Error	None
Example	<pre> >>>GET SPEC ACTIVE TIM <<<spec1 port1: pin1, pin2, pin3, pin4, pin5 >>>GET SPEC ALL TIM <<<spec1 spec2 spec3 </pre>

3.9.26 GET SPEC PERIOD

Description	The "GET SPEC PERIOD" command gives the test period in the current spec or a given spec
Syntax	GET SPEC [<specification>] PERIOD <port_name/pin_name>
Parameters	port_name: port name pin_name : pin name
Return Value	test period for the given port or pin
Return Error	None
Example	<pre> >>>GET SPEC PERIOD <port1/pin1> DFT_QUERY_ATE_INFO PORT_PERIOD port1/pin1 40 </pre>

3.9.27 GET XMODE

Description	The "GET XMODE" command gives the x mode for a given pin under current primary or a given timing specification
Syntax	GET XMODE <port_name>/<pin_name>
Parameters	pin_name: pin name port name: port name
Return Value	An integer number
Return Error	Condition: the pin_name does not exist Return: ERROR: Pin does not exist
Example	<pre> >>>GET XMODE <port_name>/<pin_name> <<<1 </pre>

3.9.28 GET DVCMAP

Description	The " GET DVCMAP" command gives the DVC mapping info
Syntax	GET DVCMAP <pin_name>
Parameters	pin_name: the pin name in Waveform definition
Return Value	Waveform definition <pin> <device_cycle_name> <physical_waveform_index> <definition_string>

Return Error	Condition: the specified pin name does not exist in Waveform definition Return: WARNING: no results is found
Example	<pre>>>>GET DVCMAP pin1 <<<pin1 0 0 {d1:!Z d2:F0N} pin1 1 1 {d1:!Z d2:F1N}</pre>

3.9.29 GET WAVETBL

Description	The " GET WAVETBL" command gives the mapping wavetable name of a given port in use.
Syntax	GET WAVETBL <port>
Parameters	port: the port name
Return Value	The relevant wavetable name of the given port
Return Error	Condition: port name does not exist Return: WARNING: no results is found
Example	<pre>>>>GET WAVETBL port1 <<<Wvt_JTAG</pre>

3.10 Query Execution Results

3.10.1 GET FAILC

Description	The "GET FAILC" command gives failing cycle information with start cycle and no of fails. Before using this command, EXE command must be executed. Details please see EXE command
Syntax	GET [FAILC] [FAILC_CNT] [FAILC_PIN] [<pinlist>] [STARTC=<spos>] [NO_OF_CYCLES=n>]
Parameters	pinlist: pin list from which the result will be retrieved, separated by "," spos : cycle number from the beginning of the pattern execution n : number of fails"
Return Value	pin cycle@data
Return Error	None
Example	>>>GET FAILC jtag_tdo STARTC=0 NO_OF_CYCLES=100

3.10.2 GET FAILC TAG

Description	The "GET FAILC TAG" command gives failing cycle information with start cycle and no of fails with tag. Before using this command, EXE command must be executed. Details please see EXE command.
Syntax	GET [FAILC] [FAILC_CNT] [FAILC_PIN] [TAG] [<pinlist>] [STARTC=<spos>] [NO_OF_CYCLES=n>]
Parameters	pinlist: pin list from which the result will be retrieved, separated by "," spos : cycle number from the beginning of the pattern execution n : number of fails
Return Value	pin tag@cycle@data
Return Error	None
Example	>>>GET FAILC TAG STARTC=10 NO_OF_CYCLES100 >>>GET FAILC TAG pin1,pin2 STARTC=10 NO_OF_CYCLES100

3.11 Check Command

3.11.1 CHK PAT

Description	The "CHK PAT" command checks the pattern <label> does exist.
Syntax	CHK PAT <label>
Parameters	label: multi-port burst label or single-port label or main label
Return Value	Pattern label exist. Pattern label does not exist.
Return Error	None
Example	>>>CHK PAT my_pattern_label <<<Pattern my_pattern_label does not exist.

3.11.2 CHK MEM

Description	The "CHK MEM" command checks the available free vector memory under a given <port>.
Syntax	CHK MEM <port>
Parameters	port: the port name
Return Value	Number
Return Error	None
Example	>>>CHK MEM pJTAG <<<111861664

3.11.3 CHK SHELL_PROTOCOL

Description	The "CHK SHELL_PROTOCOL" command returns the currently used SmartShell command revision.
Syntax	CHK SHELL_PROTOCOL
Parameters	None
Return Value	Protocol Number
Return Error	None
Example	>>>CHK SHELL_PROTOCOL <<<3

3.11.4 CHK PAT TYPE

Description	The "CHK PAT TYPE" command checks the pattern type.
Syntax	CHK PAT <label> TYPE
Parameters	label: any label name
Return Value	Pattern Type burst or main , muti-port or single-port
Return Error	None
Example	>>>CHK PAT jtag_demo TYPE <<<burst multi-port

3.12 Setup Selection

3.12.1 USE OPSEQ

Description	The "USE OPSEQ" command sets the primary <label> for the pattern execution. The primary label can either be a multi-port burst label or single port burst label or main label. It is multi-port burst label when the primary timing is multi-port timing; it is single-port burst label when the primary timing is a single-port multi-port timing; it is main label: only when the primary timing is a single-port timing.
Syntax	USE OPSEQ <label>
Parameters	label: label can either be multi-port burst label or single port burst label or main label.
Return Value	pass no error
Return Error	ERROR: because pattern not found"
Example	>>>USE OPSEQ my_multiport_burst_label <<<pass

3.12.2 USE SPEC

Description	The "USE SPEC" command sets a level or timing equation or a timing specification for multi-port timing set as primary.
Syntax	USE SPEC [LEV]/[TIM] [eqnno,specnu,setno] [<specification>@<setno,+>]
Parameters	<i>SETS based level or timing:</i> eqnno: the number of the equation defined in level or timing equation setup specno: the number of the spec defined by SpecTool setno: the number of either as LEVELSET or TIMINGSET <i>Multi-port based timing:</i> Specification: defined in SpecTool setno: the number of TIMINGSET for each port only one setno all ports use the same TIMINGSET #
Return Value	pass no error
Return Error	ERROR: because spec not found
Example	USE SPEC LEV 1,1,1 USE SPEC TIM 1,2,1 USE SPEC TIM specs@1,1

3.12.3 USE SETUP

Description	The "USE SETUP" command sets a level or timing equation or timing specification for multi-port timing set as primary.
Syntax	USE SETUP SPEC=<> OPSEQ=<> TEST=<>
Parameters	<i>SETS based level or timing:</i> eqnno: the number of the equation defined in level or timing equation setup

	specno: the number of the spec defined by SpecTool setno: the number of either as LEVELSET or TIMINGSET <i>Multi-port based timing:</i> Specification: defined in SpecTool setno: the number of TIMINGSET for each port only one setno is all ports use the same TIMINGSET # <i>Specifications and Operating Sequence based setup:</i> SPEC: specification <level>@<timing> OPSEQ: multiple pattern label TEST: name of the test"
Return Value	pass no error
Return Error	ERROR: because spec not found
Example	USE SETUP SPEC=1,1,1@1,2,1 OPSEQ=jtag_burst TEST=jtag_test

3.12.4 USE SUITE

Description	"The "USE SUITE" command sets the needed primaries for the given testsuite
Syntax	USE SUITE <suite_name>
Parameters	suite_name: the testsuite for which the primaries have to be set
Return Value	DFT_CHANGE_SUITE suite_name "DONE"
Return Error	Condition: the specified suite_name does not exist Return: ERROR: Can not find test suite suite_name in current test flow!
Example	>>>USE SUITE LV <<<DFT_CHANGE_SUITE LV "DONE"

3.12.5 USE PROTOCOL

Description	The "USE PROTOCOL" command selects the vision number of supported SmartShell command protocol
Syntax	USE PROTOCOL <version_number>
Parameters	version_number: it can be 2 or 3
Return Value	Pass
Return Error	None
Example	>>>USE PROTOCOL 3 <<<pass

3.13 Execution Commands

3.13.1 EXE

Description	The "EXE" command executes either main label or burst label or operating sequence main label can be recreated by set of SET PAT CONSTR commands
Syntax	EXE [<[FAILC] [FAILC_CNT] [FAILC_PIN]>] [TAG] [MAIN=<main label>]/[OPSEQ=<burst labe>] [SIGNAL=<pinlist>] [STARTC=<spos>] [NO_OF_CYCLES=n]
Parameters	FAILC : failing cycle FAILC_CNT : total fail count of all pins FAILC_PIN : failing count per pin REUSE : main label to be reused TAG : get vector comment text at failing cycle MAIN : list of main label separated by “,” main label : list of main labels OPSEQ : burst label or operating sequence burst label : one more burst labels pinlist : pin list from which the result will be retrieved, separated by “,” spos : cycle number from the beginning of the pattern execution n : number of fails
Return Value	Main label execution <i>without options:</i> pass or fail; <i>with option:</i> failing info. Operating sequence or burst label execution: <i>without options:</i> pass or fail; failing cycle and failing count can be retrieved by GET FAILC command; <i>with options:</i> pass or fail and failing information and failing cycle; <i>Format:</i> pass or fail ATE_FAILC cycle# pin expected result ATE_FAILC 14958 gpio_78 HHLLL LLLLL
Return Error	None
Example	>>>EXE FAILC <<<ATE_RES F <<<ATE_FAILC 14958 gpio_78 HHLLL LLLLL <<<ATE_FAILC 14959 gpio_78 HHLLL LLLLL

3.13.2 EXE FLOW

Description	The "EXE FLOW" command executes the flow already loaded or loads and executes an existing testflow
--------------------	---

Syntax	EXE FLOW
Parameters	None
Return Value	suite_name P/F/bypassed
Return Error	None
Example	<pre><<<EXE FLOW >>>suite1 P >>>suite2 P >>>suite3 bypassed >>>suite4 F</pre>

3.13.3 EXE TO_SUITE

Description	The "EXE TO_SUITE" command executes from the first suite to the specified testsuite, the following test suites are bypassed. With option HOLD, SmartTest keeps the HOLD status on the specified test suite and all pins are kept connected. This is mainly used to do some other actions after the execution. User need to send command EXE TO_SUITE CONTINUE to continue the flow execution. With option CONTINUE, the flow execution is finished, the HOLD status is unset.
Syntax	EXE TO_SUITE <suite_name>
Parameters	suite_name: an existing testsuite name
Return Value	DFT_EXE_TO_SUITE demo_suite "DONE" The executed suites and the results
Return Error	Condition: the specified suite_name does not exist Return: ERROR: SYNTAX_ERROR: - command : DFT_EXEC_TO_SUITE test_suite_name
Example	<pre><<<EXE TO_SUITE LV >>>DFT_EXE_TO_SUITE demo_suite "DONE" >>>suite1 P >>>suite2 F</pre>

3.13.4 EXE SUITE

Description	The "EXE SUITE" command executes an existing testsuite
Syntax	EXE SUITE <suite>
Parameters	suite: an existing testsuite name
Return Value	ATE_RES P or ATE_RES F
Return Error	Condition:if suite name does not exist Return:
Example	<pre><<<EXE SUITE suite_demoo >>>ATE_RES P</pre>

3.14 EDF Control

3.14.1 SET EDF_ON

Description	The "SET EDF_ON" command starts to record the edf log
Syntax	SET EDF_ON
Parameters	None
Return Value	Pass
Return Error	None
Example	>>>SET EDF_ON <<<pass

3.14.2 SET EDF_OFF

Description	The "SET EDF_OFF" command stops to record the edf log
Syntax	SET EDF_OFF
Parameters	None
Return Value	pass
Return Error	None
Example	>>>SET EDF_OFF <<<pass

3.14.3 GET EDF

Description	The "GET EDF" command gives the recorded edf info
Syntax	GET EDF
Parameters	None
Return Value	the recorded edf info
Return Error	Condition: you use this command before do not use command "SET EDF_ON" Return: WARNING: no results is found
Example	>>>GET EDF <<<DFT_QUERY_ATE_INFO EDF ConfigurationChangedEventType with 5 attributes received => received =>Attribute type: 0 44: TCK () 45: TRSTN () 3: all_inputs (34 pins): 0: BADD0 ()

3.15 Site Control

3.15.1 GET SITE_FOCUS

Description	The " GET SITE_FOCUS" command gives the focused site number
Syntax	GET SITE_FOCUS
Parameters	None
Return Value	the focus site
Return Error	None
Example	>>>GET SITE_FOCUS <<<2

3.15.2 SET SITE_FOCUS

Description	The " SET SITE_FOCUS" command sets the focused site (the site should have been enabled)
Syntax	SET SITE_FOCUS <site_number>
Parameters	None
Return Value	pass no error
Return Error	Error message the given <site_number> is out of range
Example	>>>SET SITE_FOCUS 2 <<<pass

3.15.3 SET SITE_ENABLE ALL

Description	The "SET SITE_ENABLE ALL" command enables all the sites
Syntax	SET SITE_ENABLE ALL
Parameters	None
Return Value	pass no error
Return Error	None
Example	>>>SET SITE_ENABLE ALL <<<pass

3.15.4 SET SITE_ENABLE

Description	The "SET SITE_ENABLE" command enables and focus on the specified site
Syntax	SET SITE_ENABLE <ALL site_id>
Parameters	"ALL" or site_id
Return Value	pass no error
Return Error	None
Example	>>>SET SITE_ENABLE <<<pass

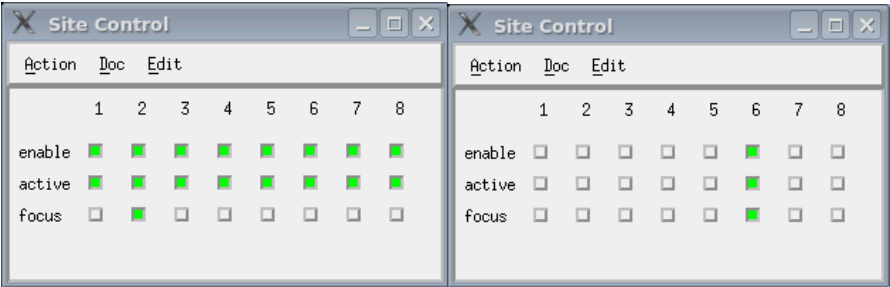


Fig. 4: Site control pop-up window before and after the command

3.15.5 SET SITE_DISABLE

Description	The "SET SITE_DISABLE" command disables all the sites or the specified site
Syntax	SET SITE_DISABLE <ALL site_id>
Parameters	"ALL" or site_id
Return Value	pass no error
Return Error	None
Example	>>>SET SITE_DISABLE ALL <<<pass

3.16 Suite Control

3.16.1 GET SUITE SETUP <suite_name>

Description	The " GET SUITE SETUP" command gives all the setup info or the setup info of the specified suite
Syntax	GET SUITE SETUP <suite_name>
Parameters	suite_name: the suite name defined in the test flow
Return Value	Suite setup info
Return Error	Condition: the suite dose not exist Return: Suite Default does not exist
Example	>>>GET SUITE SETUP jtag_demo <<<timing info <<<spec: "spec_pJTAG_STD_spec" <<<timset: "1" <<<level info <<<eqn: 1 <<<spec: 1 <<<levelset: 1 <<<pattern info <<<pattern: jtag_demo

3.16.2 GET SETUP_FILE

Description	The " GET SETUP_FILE" command gives the loaded setup files
Syntax	GET SETUP_FILE
Parameters	None
Return Value	The loaded setup files
Return Error	None
Example	>>>GET SETUP_FILE <<<DFT_QUERY_ATE_INFO SETUP_FILE <<<testflow : smartshell_demo.flw <<<pin : smartshell_demo.pin <<<level : smartshell_demo.lvl <<<timing : smartshell_demo.tim.mfh <<<pattern : smartshell_demo.pmf1

3.16.3 DELETE SUITE

Description	The " DELETE SUITE" command deletes the given suite
Syntax	DELETE SUITE <suite_name>
Parameters	suite_name: existing suite name
Return Value	None
Return Error	None
Example	>>>DELETE SUITE jtag_demo <<<pass

3.17 COMMAND RECORD

3.17.1 LOG CMD

Description	The "LOG CMD" command defines the prefix string of unrecord commands, then the relevant commands will be recorded in the log files.
Syntax	LOG CMD [ALL] [<cmd_prefix_name>]
Parameters	The first word of command, like "GET", "SET" and so on
Return Value	None
Return Error	None
Example	>>>LOG CMD GET <<<pass

3.17.2 LOG STR

Description	The "LOG STR" command records the string in the log file but do nothing
Syntax	LOG STR <string>
Parameters	string: any string
Return Value	Pass
Return Error	None
Example	>>>LOG STR start to debug <<<pass

3.17.3 UNLOG CMD

Description	The "UNLOG CMD" command defines the prefix string of unrecord commands, then the relevant commands will not be recorded in the log files.
Syntax	UNLOG CMD [ALL] [<cmd_prefix_name>]
Parameters	The specified command.
Return Value	Pass
Return Error	None
Example	>>>UNLOG CMD GET <<<pass